

CEFET/RJ

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca

Bacharelado em Ciência da Computação

GCC1734 – Inteligência Artificial

Aprendizado por Reforço

Nota de Aula 6

Prof. Eduardo Bezerra

Escola de Informática e Computação

CEFET/RJ

8 de outubro de 2026

Sumário

1	Motivação e Contexto	4
1.1	O que é aprendizado por reforço?	4
1.2	Hipótese de recompensa	5
1.3	Relação com outros paradigmas de IA	5
2	Processo de Decisão de Markov (PDM)	6
2.1	Formulação formal	6
2.2	Propriedade de Markov	8
2.3	Episódios e interação com o ambiente	8
2.4	Exemplo: Mundo Grade	8
3	Política e Utilidade	10
3.1	Política e política ótima	10
3.2	Retorno e fator de desconto	11
3.2.1	Exemplo resolvido	11
3.3	Valores de estados e de ações	12
3.3.1	Valor de um estado	13
3.3.2	Valor de uma ação	13
3.3.3	Dos valores à política	13
3.4	Equações de Bellman de otimalidade	14
3.4.1	Do retorno completo ao primeiro passo	14
3.4.2	Avaliando uma ação	14
3.4.3	Escolhendo a melhor ação	15
3.4.4	Interpretando a estrutura recursiva	15
3.4.5	Exemplo resolvido no Mundo Grade	16
3.4.6	Das equações ao aprendizado	17
4	Aprendizado por Diferença Temporal	18
4.1	Três formas de aprender valores	18
4.2	O erro de Diferença Temporal	19
4.3	TD e a equação de Bellman	20
5	Métodos Tabulares de Diferença Temporal	20
5.1	Q-learning	20
5.1.1	Ideia central	20
5.1.2	Regra de atualização	22
5.1.3	Exemplo resolvido	23
5.1.4	Taxa de aprendizado	24
5.1.5	Convergência e propriedades	24
5.1.6	Pseudocódigo	25
5.2	SARSA	25
5.2.1	Regra de atualização	25
5.2.2	On-policy versus off-policy: intuição	26
5.2.3	Pseudocódigo	27
5.3	Exemplo resolvido: comparação entre Q-learning e SARSA	29

6	Exploração versus Aproveitamento	30
6.1	O dilema fundamental	30
6.2	Estratégia ε -greedy	31
6.2.1	Exemplo resolvido	31
6.3	Estratégia softmax (Boltzmann)	32
6.3.1	Exemplo resolvido	33
6.4	Estratégia por funções de exploração	33
6.4.1	Exemplo resolvido	34
7	Q-Learning com Aproximação Linear	35
7.1	Limitações do Q-learning tabular	35
7.2	Características e representação	36
7.3	Engenharia de características: o verdadeiro gargalo	37
7.4	De características de estado para características de estado-ação	39
7.5	Por que normalizar as características?	40
7.6	A Q-função aproximada	41
7.7	Exemplo: projetando e refinando características no Mundo Grade	42
7.8	Regra de atualização dos pesos	45
7.8.1	Exemplo resolvido	45
7.9	Relação com regressão linear e erro quadrático	46
7.10	O ônus e o bônus da aproximação linear	47
8	Reward Shaping e Recompensas Intermediárias	48
8.1	Por que pensar em <i>reward shaping</i> ?	48
8.2	A ideia básica	49
8.3	<i>Reward shaping</i> baseado em potencial	50
8.4	Exemplo no Mundo Grade	51
8.5	Limitações	53
9	Aprendizado por Reforço com Feedback Humano	54
9.1	De preferências humanas a uma função de recompensa	54
9.2	O processo de RLHF	55
9.3	RLHF como um problema de aprendizado por reforço	56
9.4	Exemplo resolvido: aprendizado de preferências sobre respostas	56
9.5	Limitações e cuidados	59
9.6	RLHF e <i>reward shaping</i>	59
10	Resumo	60
11	Atividades Intra-Classe	62
11.1	Aquecimento	62
11.2	Q-learning tabular no Mundo Grade	63
11.3	Comparação entre Q-learning e SARSA	63
11.4	Q-learning com aproximação linear	64
11.5	Exploração e arrependimento	64
11.6	Perguntas para discussão em sala	64

Objetivos

Ao final da leitura, você deve ser capaz de:

Motivação e formulação

- Explicar o que é aprendizado por reforço e em que difere dos paradigmas de busca e de aprendizado supervisionado.
- Formular um problema de aprendizado por reforço como um Processo de Decisão de Markov (PDM) parcialmente especificado.

PDM e política ★

- Identificar os componentes de um PDM (S, A, T, R, γ) e explicar por que T e R são desconhecidos no AR.
- Definir política ótima e distingui-la de um plano.
- Definir $V^*(s)$ e $Q^*(s, a)$ e explicar a relação entre eles por meio da equação de Bellman.

Diferença Temporal ★

- Explicar o que é o aprendizado por Diferença Temporal (TD) e em que difere de Monte Carlo e de Programação Dinâmica.
- Definir o erro de TD δ_t e interpretar seu sinal como indicador de “surpresa” causada por uma transição.
- Reconhecer Q-learning e SARSA como dois algoritmos TD que diferem apenas no alvo de atualização.

Q-learning e SARSA ★

- Enunciar as regras de atualização do Q-learning e do SARSA e executá-las manualmente sobre uma sequência de transições.
- Explicar a distinção *off-policy* (Q-learning) versus *on-policy* (SARSA) e suas consequências práticas.
- Explicar o papel da taxa de aprendizado α e do fator de desconto γ .
- Descrever o dilema exploração-aproveitamento e contrastar as estratégias ϵ -greedy e funções de exploração.

Aproximação linear ★

- Descrever como características (*features*) permitem generalizar o aprendizado para estados não visitados.
- Enunciar a regra de atualização dos pesos no Q-learning com aproximação linear e executá-la sobre um exemplo concreto.

Nesta leitura, você verá primeiro o que é aprendizado por reforço e como ele se relaciona com os algoritmos de busca e de planejamento já estudados (Seção 1). Em seguida, a formalização do problema via Processos de Decisão de Markov (Seção 2) e os conceitos de política, utilidade e valores de estado e de estado-ação (Seção 3). Depois, o arcabouço de Diferença Temporal (TD), que situa o Q-learning e o SARSA em uma família mais ampla de métodos (Seção 4). A seguir, o algoritmo Q-learning tabular, o SARSA, a distinção *off-policy/on-policy*, a regra de atualização e o dilema exploração-aproveitamento (Seções 5–6). Por fim, as limitações do Q-learning tabular e como a aproximação linear as supera (Seção 7).

Os objetivos marcados com $\star$ são os mais centrais para a aula: concentre-se neles se o tempo for limitado.

1 Motivação e Contexto

1.1 O que é aprendizado por reforço?

Nos algoritmos de busca estudados até aqui (busca em grafos, Minimax e MCTS) o agente assume acesso completo ao modelo do ambiente: ele dispõe de uma função de transição e consegue raciocinar sobre as consequências de suas ações, seja por meio de uma função de avaliação, seja por simulação. O problema do agente é *encontrar* a política de ações que maximiza algum critério, dado esse modelo completo.

O aprendizado por reforço (AR, ou *reinforcement learning*) estuda como aprender a agir para maximizar recompensas acumuladas a partir da experiência. Há AR em ambientes determinísticos ou estocásticos, com dinâmica conhecida ou desconhecida. Nesta nota, estudamos principalmente um cenário *livre de modelo*, no qual o agente *não recebe o modelo do ambiente*. Ele sabe quais ações pode executar, mas não sabe *a priori* quais são as consequências de cada ação nem quais recompensas elas produzem. Seu único recurso é interagir com o ambiente, executar ações, observar o que acontece e receber um sinal numérico de *feedback* denominado *recompensa*, e aprender, progressivamente, qual comportamento é desejável.

A pergunta central do cenário livre de modelo estudado aqui pode ser formulada assim: *como um agente racional aprende a tomar boas decisões apenas interagindo com o ambiente, sem acesso ao modelo que governa esse ambiente?*

A cada interação, o agente percorre o seguinte ciclo, ilustrado na Figura 1:

1. Observa o estado atual s do ambiente.
2. Seleciona e executa uma ação a .
3. Recebe uma recompensa numérica r e observa o próximo estado s' .

Esse ciclo se repete por muitas iterações (potencialmente milhões), ao longo das quais o agente deve identificar qual ação é adequada em cada estado; ou seja, aprender uma *política*.

Uma criança aprendendo a andar de bicicleta não recebe um manual com a função de transição “se você inclinar X graus e pedalar com força Y , você avançará Z centímetros”. Ela tenta, cai (recompensa negativa), ajusta seu comportamento, e gradualmente aprende quais posturas e movimentos produzem o resultado desejado. Esse é exatamente o paradigma do AR: aprender por tentativa e erro a partir de um sinal de *feedback*. Embora o AR pareça simples à primeira vista, ele esconde uma dificuldade fundamental: o agente não observa diretamente o “valor” de uma ação. Ele observa apenas recompensas locais, frequentemente ruidosas, e consequências indiretas que podem se manifestar muitos

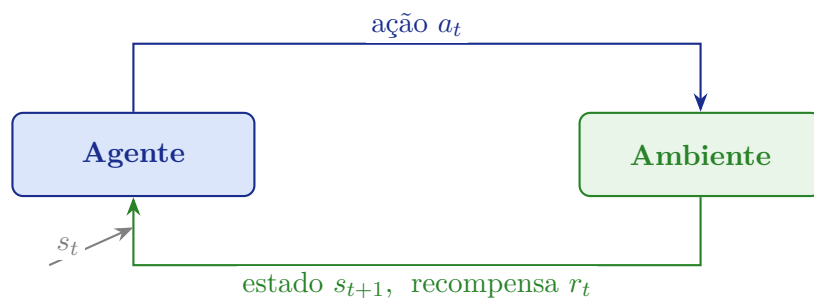


Figura 1: Ciclo de interação entre o agente e o ambiente no aprendizado por reforço. A cada passo de tempo t , o agente observa o estado atual s_t , executa uma ação a_t e recebe do ambiente a recompensa r_t e o próximo estado s_{t+1} . O objetivo do agente é aprender uma política que maximize a soma esperada de recompensas ao longo do tempo.

passos no futuro.

O desafio central do AR é o chamado *problema de atribuição de crédito* (*credit assignment problem*): descobrir *quais ações passadas foram responsáveis pelas recompensas obtidas agora*. Se o agente vence um jogo de xadrez após 80 lances, qual lance foi decisivo? Todos contribuíram igualmente? O AR precisa responder a essa pergunta a partir de experiência, sem acesso ao modelo do ambiente.

1.2 Hipótese de recompensa

O AR é construído sobre uma premissa fundamental denominada *hipótese de recompensa*: qualquer objetivo que o agente deva alcançar pode ser formalizado como a maximização de uma recompensa cumulativa esperada. Toda meta pode ser codificada em um único sinal numérico recebido a cada passo de tempo, como nos exemplos a seguir: navegar até um destino, jogar bem um jogo de tabuleiro, ou controlar um robô.

Essa hipótese é ao mesmo tempo poderosa e conveniente: ela unifica problemas de naturezas muito diferentes sob um único *framework* matemático. Ela é também uma aposta: o projetista do sistema deve garantir que o sinal de recompensa captura adequadamente o que se quer que o agente aprenda. Uma função de recompensa mal projetada pode levar o agente a comportamentos indesejados, mesmo que ele a maximize perfeitamente.

1.3 Relação com outros paradigmas de IA

É útil situar o AR em relação aos paradigmas estudados anteriormente. A tabela abaixo resume as diferenças mais relevantes.

Paradigma	Modelo do ambiente	do Feedback disponível	dis-	O que o agente aprende
Busca (A*, UCS)	Completo	Nenhum		Plano ótimo
Minimax MCTS	/ Completo (re-gras do jogo)	Resultado do jogo		Política adversarial
Aprendizado supervisionado	Não se aplica	Rótulos corretos		Função de previsão
Aprendizado por reforço	Desconhecido	Recompensa escalar		Política ótima

Na busca determinística, o agente conhece as consequências das ações e procura um plano usando esse modelo. No cenário livre de modelo desta nota, o modelo de transição e as recompensas não estão disponíveis de antemão; o agente aprende valores ou políticas a partir da experiência, sem estimar explicitamente esse modelo.

Pergunta 1

O AR é frequentemente descrito como aprendizado por “tentativa e erro”. Explique o que essa expressão significa no contexto formal do AR: o que o agente “tenta”, o que constitui um “erro” e como ele aprende a partir dele. Construa um exemplo simples com um agente em uma grade 2×2 com duas ações possíveis, descrevendo uma sequência de três transições e indicando o que o agente pode concluir de cada uma.

2 Processo de Decisão de Markov (PDM)

O arcabouço matemático padrão para formular problemas de AR é o *Processo de Decisão de Markov* (PDM, ou *Markov Decision Process*, MDP). Um PDM captura formalmente a ideia de um agente que interage sequencialmente com um ambiente estocástico, recebendo recompensas e transitando entre estados.

2.1 Formulação formal

Um PDM é definido pela tupla:

$$\langle S, A, T, R, \gamma \rangle$$

onde:

- S é o conjunto de estados do ambiente, incluindo um estado inicial s_0 .
- A é o conjunto de ações disponíveis para o agente (em geral, pode variar por estado).
- $T(s, a, s')$ é o *modelo de transição*: a probabilidade de o ambiente transitar para o estado s' quando o agente executa a ação a no estado s , isto é, $T(s, a, s') = \Pr(s' \mid s, a)$. Para cada par (s, a) , a soma sobre todos os s' possíveis é 1.

- $R(s, a, s')$ é a *função de recompensa*: o valor numérico recebido pelo agente após executar a em s e transitar para s' . Em formulações simplificadas, escreve-se $R(s)$. Nas seções seguintes, adotaremos essa forma simplificada, pois a recompensa no Mundo Grade depende apenas do estado alcançado.
- $\gamma \in [0, 1)$ é o *fator de desconto* no cenário descontado adotado aqui, que controla a importância relativa de recompensas futuras em relação às imediatas (detalhado na Seção 3.2).

Ao olhar para a tupla $\langle S, A, T, R, \gamma \rangle$, adotaremos as seguintes hipóteses sobre o que o agente *sabe* e o que ele *não sabe* no cenário tabular estudado:

- **O agente conhece:** o conjunto de estados S e o conjunto de ações A .
- **O agente não conhece:**
 - quais estados são bons ou ruins (não sabe os valores $R(s)$ antes de visitá-los);
 - quais ações levam a quais estados (não sabe $T(s, a, s')$);
 - quais recompensas cada ação produzirá em cada estado.

O agente aprende valores ou políticas a partir de amostras de experiência (s, a, r, s') coletadas pela interação com o ambiente, sem precisar estimar T e R explicitamente. No Mundo Grade estocástico, essas amostras são ruidosas. Conhecer previamente todos os estados e desconhecer T e R são hipóteses deste exemplo, não requisitos gerais do AR ou dos métodos livres de modelo. A tabela abaixo compara os componentes de um PDM com os de um problema de busca monoagente:

Componente	Busca monoagente	PDM (AR)
Estados	S	S (o mesmo)
Ações	$ACTIONS(s)$	A (o mesmo)
Transição	$T(s, a)$ determinística	$T(s, a, s')$ probabilística
Custo	$c(s, a)$	$R(s, a, s')$ (recompensa)
Terminal	$GoalTest(s)$	estados terminais em S
Objetivo	minimizar custo	maximizar recompensa cumulativa
Solução	plano = sequência de ações	política $\pi(s)$

Em um PDM *completo*, o agente conhece T e R antes de agir e pode calcular a política ótima diretamente por algoritmos como Iteração de Valor. No cenário de AR *livre de modelo* adotado nestas notas, o agente recebe um PDM *parcialmente especificado*: ele conhece S e A , mas não conhece T nem R . Ele deve aprender a política ótima a partir da experiência. (Em abordagens baseadas em modelo, o agente estima T e R explicitamente durante o aprendizado, o que foge ao escopo desta aula.)

Pergunta 2

Com base na tabela acima, identifique as principais diferenças entre a formulação de busca monoagente e a formulação PDM. Para cada diferença, explique o que o componente novo ou modificado captura que a formulação monoagente não conseguia representar.

2.2 Propriedade de Markov

O nome “Markov” no PDM refere-se à *propriedade de Markov*: o estado seguinte depende exclusivamente do estado atual e da ação tomada, e não do histórico completo de estados e ações passados. Formalmente:

$$\Pr(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \Pr(s_{t+1} \mid s_t, a_t)$$

Essa propriedade é análoga à que utilizamos nos algoritmos de busca, onde a função sucessora depende apenas do estado e ação atuais. Ela torna o estado uma representação *suficiente* do passado para a tomada de decisão: um agente que observe apenas o estado atual tem toda a informação necessária para agir otimamente, desde que o estado seja projetado de forma adequada.

Em conjunto, S e A descrevem as situações e decisões possíveis; T e R descrevem, respectivamente, a dinâmica e o feedback do ambiente; e γ especifica como o retorno futuro é ponderado.

2.3 Episódios e interação com o ambiente

Um **episódio** é uma sequência finita de interações entre o agente e o ambiente, com início em um estado s_0 e término quando um estado terminal é atingido. Formalmente:

$$s_0 \xrightarrow{a_0, r_0} s_1 \xrightarrow{a_1, r_1} s_2 \xrightarrow{a_2, r_2} \dots \xrightarrow{a_{T-1}, r_{T-1}} s_T \quad (s_T \text{ terminal})$$

O agente aprende progressivamente a partir de múltiplos episódios sucessivos: ao final de cada episódio, o ambiente é reiniciado e um novo começa. A intuição de “propagação de valor” que veremos na Seção 5 faz referência diretamente a esses episódios: valores de estado próximos ao terminal convergem em poucos episódios; estados distantes exigem mais.

2.4 Exemplo: Mundo Grade

Para ilustrar os componentes de um PDM, utilizaremos o *Mundo Grade* (*Grid World*), um ambiente simples e amplamente usado na literatura de AR. O Mundo Grade é uma grade 4×3 com as seguintes características:

- O agente ocupa uma célula da grade e pode mover-se nas quatro direções cardeais: Norte, Sul, Leste, Oeste.
- Há dois estados terminais: uma saída com recompensa $+1$ (objetivo desejado), localizada em $(4, 3)$, e um precipício com recompensa -1 (objetivo a evitar), localizado em $(4, 2)$. A célula $(2, 2)$ é bloqueada (obstáculo intransponível).
- As transições são *estocásticas*: quando o agente tenta mover-se para uma direção, ele consegue com probabilidade $0,8$; com probabilidade $0,1$ desloca-se 90° à esquerda; e com probabilidade $0,1$, 90° à direita. Movimentos que levariam o agente a uma parede ou ao obstáculo mantêm-no na posição atual.
- A recompensa por passo de tempo em estados não terminais é um valor pequeno (denominado *living reward*), tipicamente negativo, que incentiva o agente a buscar os estados terminais sem demora.

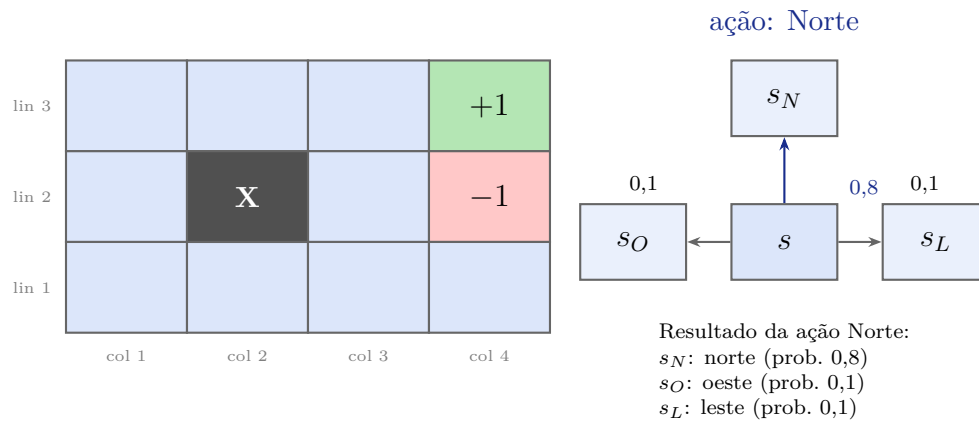


Figura 2: Esquerda: o Mundo Grade 4×3 . A célula verde na posição (4,3) representa o estado terminal com recompensa +1; a célula vermelha em (4,2), o estado terminal com recompensa -1; a célula preta (X) em (2,2) é o obstáculo intransponível; as demais células são estados não terminais. Direita: probabilidades de transição ao executar a ação Norte a partir de um estado s sem obstáculos adjacentes. O agente chega ao estado norte (s_N) com probabilidade 0,8 e é desviado para o estado oeste (s_O) ou leste (s_L) com probabilidade 0,1 cada.

A Figura 2 ilustra o Mundo Grade e as probabilidades de transição para a ação Norte. Os componentes do PDM para o Mundo Grade são:

- S : as 11 células não bloqueadas da grade (incluindo os dois estados terminais).
- A : {Norte, Sul, Leste, Oeste} em estados não terminais.
- $T(s, a, s')$: a função estocástica descrita acima. Por exemplo, considerando o estado (1,1) (canto inferior esquerdo) e a ação Norte: o agente tenta mover-se para o Norte com probabilidade 0,8, chegando a (1,2); com probabilidade 0,1 desvia para Leste, chegando a (2,1); e com probabilidade 0,1 desvia para Oeste, mas como não há célula a oeste de (1,1), o agente permanece em (1,1). Portanto: $T((1,1), \text{Norte}, (1,2)) = 0,8$; $T((1,1), \text{Norte}, (2,1)) = 0,1$; $T((1,1), \text{Norte}, (1,1)) = 0,1$.
- $R(s)$: +1 no estado terminal positivo; -1 no estado terminal negativo; um valor pequeno (e.g., -0,04) em todos os demais estados.

Transições estocásticas capturam a incerteza do mundo real: um robô que recebe o comando “andar para frente” pode escorregar ou ter um motor com folga. A estocasticidade força o agente a planejar considerando *todos* os resultados possíveis de cada ação. No Mundo Grade, ela é modelada como 20% de ruído perpendicular à direção pretendida.

Pergunta 3

Considere um agente no Mundo Grade que experimenta a seguinte sequência de transições a partir de (1,1): Norte $\rightarrow$ (1,2), $r = -0,04$; Norte $\rightarrow$ (1,3), $r = -0,04$; Leste $\rightarrow$ (2,3), $r = -0,04$; Leste $\rightarrow$ (3,3), $r = -0,04$; Leste $\rightarrow$ (4,3) (Terminal⁺), $r = +1$. Calcule a recompensa total não descontada e a recompensa descontada com $\gamma = 0,9$. Interprete a diferença entre os dois valores.

Nas próximas seções, reutilizaremos o Mundo Grade para ilustrar:

- políticas ótimas;
- cálculo de valores $V(s)$ e $Q(s, a)$;
- atualizações do Q-learning e do SARSA;
- estratégias de exploração.

A ideia é usar um mesmo ambiente para aprofundar progressivamente os conceitos.

3 Política e Utilidade

O PDM descreve os estados, as ações, as transições e as recompensas do ambiente. Resta estabelecer como o agente deve agir e como avaliar suas escolhas. Nesta seção, introduzimos a política como regra de decisão e o retorno como medida das consequências acumuladas dessas decisões. A partir desses conceitos, definimos os valores de estados e de ações e apresentamos as equações de Bellman, que relacionam a recompensa imediata ao valor da continuação da interação.

3.1 Política e política ótima

A solução de um PDM é uma *política* π , que especifica como o agente deve agir em cada estado. Uma política *determinística* associa a cada estado não terminal uma ação disponível, $\pi(s) \in A(s)$. Mais geralmente, uma política *estocástica* especifica uma distribuição de probabilidades $\pi(a | s)$ sobre as ações disponíveis. Nesse caso, o agente sorteia uma ação segundo essa distribuição. Por exemplo, no Mundo Grade, uma política determinística indica qual ação executar em cada célula não terminal.

Uma política é especialmente útil em ambientes estocásticos: mesmo executando uma ação determinada, o agente pode chegar a diferentes estados sucessores. Uma sequência fixa de ações não se adapta a esses resultados. A política, por sua vez, especifica o comportamento a partir do estado em que o agente efetivamente se encontra. Uma vez disponível, ela pode ser consultada a cada passo, sem exigir uma nova busca ou um novo planejamento.

Uma *política ótima* π^* maximiza o retorno esperado, isto é, a soma esperada das recompensas ao longo da interação, com o desconto adotado. Isso exige considerar tanto as recompensas imediatas quanto as consequências futuras das ações. Uma ação que oferece uma recompensa imediata maior pode ser menos vantajosa se levar a situações desfavoráveis nos passos seguintes.

A otimalidade se refere ao *retorno esperado*, não ao resultado de cada episódio. Em um ambiente estocástico, mesmo uma política ótima pode produzir um resultado ruim em uma execução particular.

A Figura 3 mostra uma política ótima para o Mundo Grade com os parâmetros indicados na legenda. As setas em cada estado indicam a ação recomendada por esta política em cada estado. Observe que, em $(4, 1)$, a ação recomendada é Oeste, e não Norte; executar Norte nessa célula levaria ao terminal negativo com alta probabilidade. Já em $(3, 2)$, a política recomenda Norte, embora exista o risco de um desvio para esse terminal; executar Leste neste estado tornaria esse resultado muito mais provável. Já a escolha da ação Oeste em $(3, 2)$ não oferece risco de transição para o terminal $(4, 2)$.

Assim, a política ótima considera conjuntamente as probabilidades de transição, as recompensas e o desconto. Com recompensa não terminal de $-0,04$, percursos mais longos acumulam mais penalidades, mas ainda podem ser vantajosos quando reduzem suficientemente o risco de receber a recompensa terminal -1 .

→	→	→	+1
↑	X	↑	-1
↑	→	↑	←

Figura 3: Política ótima no Mundo Grade com $\gamma = 0,9$, recompensa não terminal $-0,04$ e probabilidades de movimento $0,8/0,1/0,1$. Cada célula contém a ação recomendada. Observe que a política evita o estado -1 , mesmo que isso implique trajetórias mais longas até o objetivo positivo.

Para quantificar essa comparação, primeiro definiremos o retorno e o papel do desconto. Em seguida, introduziremos as funções de valor, que expressam o retorno esperado a partir de um estado ou de uma escolha de ação.

3.2 Retorno e fator de desconto

Para comparar políticas, precisamos de uma medida que reúna as recompensas recebidas ao longo da interação. Essa medida é o *retorno*. Na formulação descontada, o fator $\gamma \in [0, 1)$ determina o peso das recompensas conforme o número de passos até seu recebimento. Nesta nota, usamos o termo *utilidade* para designar esse retorno descontado. Para uma sequência de recompensas $r_0, r_1, r_2, \dots$, temos:

$$U = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \dots = \sum_{t=0}^{\infty} \gamma^t r_t$$

O papel de γ pode ser entendido em dois níveis:

- **Matemático:** com $0 \leq \gamma < 1$ e recompensas limitadas, $|r_t| \leq M$, a série converge absolutamente, pois $|U| \leq M/(1 - \gamma)$. Em tarefas episódicas com retorno bem definido (por exemplo, horizonte finito e recompensas limitadas), pode-se usar $\gamma = 1$.
- **Comportamental:** γ próximo de 1 (e.g., 0,99) torna o agente “paciente”, isto é, ele valoriza recompensas futuras quase tanto quanto as imediatas. γ próximo de 0 (e.g., 0,1) torna o agente “impaciente”: ele prioriza fortemente as recompensas imediatas.

3.2.1 Exemplo resolvido

Neste exemplo, vamos analisar o efeito do fator de desconto na escolha de trajetórias. Considere duas sequências de recompensas simplificadas a partir de um mesmo estado inicial. Para isolar o efeito do tempo, omitimos aqui o custo por passo do Mundo Grade:

Trajetoória A (curta):

$$r_0 = 0, r_1 = 0, r_2 = +1$$

Trajetoória B (mais longa):

$$r_0 = 0, r_1 = 0, r_2 = 0, r_3 = 0, r_4 = +1$$

Vamos comparar as utilidades descontadas para diferentes valores de γ .

Caso 1: $\gamma = 0,9$ (agente paciente)

$$U_A = \gamma^2 \cdot 1 = 0,9^2 = 0,81$$

$$U_B = \gamma^4 \cdot 1 = 0,9^4 \approx 0,66$$

Nesse caso, a trajetória curta é preferida, pois a penalização temporal ainda é moderada.

Caso 2: $\gamma = 0,5$ (agente impaciente)

$$U_A = 0,5^2 = 0,25$$

$$U_B = 0,5^4 = 0,0625$$

A diferença se torna muito mais acentuada: a trajetória longa perde valor rapidamente.

Interpretação:

- Quanto menor o valor de γ , maior a penalização de recompensas futuras;
- Para a mesma recompensa positiva, um desconto menor favorece recebê-la mais cedo;
- Sem probabilidades de fracasso e suas penalidades, não podemos comparar o risco das trajetórias.

O efeito sobre o risco depende das recompensas e transições; não há relação geral entre desconto menor e comportamento mais agressivo. No Mundo Grade desta nota, com recompensa não terminal $-0,04$, a ação ótima em $(4, 1)$ é Oeste para $\gamma = 0,9$, mas Sul para $\gamma = 0,5$: neste último caso, permanecer na borda inferior evita a penalidade imediata de subir para $(4, 2)$.

Pergunta 4

Suponha $\gamma = 0,9$. Um agente percorre um episódio de 5 passos com recompensas $r_0 = 0$, $r_1 = 0$, $r_2 = 0$, $r_3 = 0$, $r_4 = +1$. Calcule a utilidade descontada. Em seguida, repita com $\gamma = 0,5$. O que a diferença nos resultados revela sobre o papel de γ ?

Os exemplos anteriores comparam sequências específicas de recompensas. Em um ambiente estocástico, uma mesma política pode produzir diferentes sequências. Precisamos, portanto, considerar o retorno esperado. As funções de valor apresentadas a seguir expressam essa expectativa.

3.3 Valores de estados e de ações

Uma política específica como o agente deve agir. Para comparar ações e identificar uma política ótima, precisamos avaliar suas consequências: quanto retorno o agente pode esperar ao começar em determinado estado ou ao executar determinada ação?

Duas funções expressam essas avaliações: o valor de estado $V^*(s)$ e o valor de ação $Q^*(s, a)$. O asterisco indica que consideramos uma continuação ótima da interação.

3.3.1 Valor de um estado

O *valor ótimo de um estado*, $V^*(s)$, é o retorno esperado quando o agente começa em s e segue uma política ótima dali em diante. Esse retorno inclui todas as recompensas futuras, com o desconto correspondente:

$$r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

Assim, $V^*(s)$ responde à pergunta:

Quanto retorno posso esperar a partir de s , se agir otimamente?

É importante distinguir três quantidades:

- a *recompensa imediata* é recebida em uma transição;
- o *retorno de um episódio* é a soma descontada das recompensas efetivamente recebidas;
- o *valor do estado* é o retorno esperado a partir desse estado, considerando a política adotada.

No caso de V^* , a política considerada é ótima. Mesmo seguindo essa política, episódios diferentes podem produzir retornos diferentes, pois as transições do ambiente podem ser estocásticas.

3.3.2 Valor de uma ação

O *valor ótimo de uma ação*, $Q^*(s, a)$, é o retorno esperado quando o agente começa em s , executa a ação a e age otimamente a partir do próximo estado.

Portanto, $Q^*(s, a)$ responde à pergunta:

Quanto retorno posso esperar se executar a em s e agir otimamente depois?

A primeira ação está fixada: ela é a , mesmo que não seja a melhor ação disponível. A suposição de comportamento ótimo se aplica às ações seguintes.

Para representar essa distinção graficamente, podemos usar um *q-estado*: o par (s, a) representa a situação em que o agente está em s e já decidiu executar a , mas o resultado da ação ainda não ocorreu. Trata-se de uma representação auxiliar do par estado-ação, não de um novo estado físico do ambiente.

3.3.3 Dos valores à política

Se o agente pode escolher a primeira ação e agir otimamente depois, o valor do estado é o maior valor entre as ações disponíveis:

$$\boxed{V^*(s) = \max_a Q^*(s, a)} \quad \text{para estados não terminais.}$$

Uma política ótima pode, portanto, escolher qualquer ação que atinja esse máximo:

$$\pi^*(s) \in \arg \max_a Q^*(s, a).$$

Se houver empate, mais de uma ação pode ser ótima.

Essa relação explica a utilidade de aprender valores de ação: conhecendo Q^* , o agente pode escolher uma ação ótima comparando os valores das ações disponíveis no estado atual.

Ainda falta responder uma pergunta central: **como relacionar esses valores às recompensas e às transições do ambiente?**

3.4 Equações de Bellman de otimalidade

O valor de uma ação considera o retorno de toda a continuação da interação. Essa continuação pode envolver muitos passos e inúmeras trajetórias possíveis. Como avaliar a ação sem enumerar todas essas trajetórias?

A ideia é *separar o primeiro passo do restante da interação*. Ao executar uma ação, o agente recebe uma recompensa imediata e chega a um novo estado. O valor desse estado resume o retorno esperado de toda a continuação.

3.4.1 Do retorno completo ao primeiro passo

Considere uma sequência de recompensas $r_0, r_1, r_2, \dots$. Seu retorno pode ser reescrito como:

$$\underbrace{r_0 + \gamma r_1 + \gamma^2 r_2 + \dots}_{\text{retorno completo}} = r_0 + \gamma \underbrace{(r_1 + \gamma r_2 + \gamma^2 r_3 + \dots)}_{\text{retorno a partir do próximo estado}}.$$

Essa decomposição mostra que:

O retorno completo é a recompensa do primeiro passo mais o retorno da continuação, descontado por γ .

Se o agente agir otimamente a partir do próximo estado s' , o retorno esperado dessa continuação será $V^*(s')$. Para uma transição de s para s' causada pela ação a , combinamos:

$$\underbrace{R(s, a, s')}_{\text{recompensa imediata}} + \underbrace{\gamma V^*(s')}_{\text{valor descontado da continuação}}.$$

O próximo estado, porém, pode ser incerto. Precisamos considerar todos os sucessores possíveis e suas probabilidades.

3.4.2 Avaliando uma ação

Fixe um estado s e uma ação a . O ambiente determina o próximo estado de acordo com as probabilidades $T(s, a, s')$. Portanto, o valor da ação é uma média ponderada:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')].$$

Essa é uma relação de Bellman para os valores ótimos. Cada sucessor contribui com:

- sua probabilidade de ocorrência, $T(s, a, s')$;
- a recompensa imediata da transição, $R(s, a, s')$;
- o valor ótimo da continuação, descontado por γ .

Para um estado terminal, adotamos:

$$V^*(\text{terminal}) = 0.$$

A recompensa recebida ao entrar nesse estado já está incluída em $R(s, a, s')$. O valor de continuação é zero porque o episódio terminou.

3.4.3 Escolhendo a melhor ação

Para obter o valor ótimo de um estado, escolhamos a ação com maior valor. Substituindo a expressão de Q^* na relação $V^*(s) = \max_a Q^*(s, a)$, obtemos:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')].$$

Essa é a equação de Bellman de otimalidade para V^* . Sua estrutura reúne duas operações distintas:

- *média ponderada sobre os sucessores*: o agente não escolhe o resultado da transição;
- *máximo sobre as ações*: o agente escolhe qual ação executar.

Em outras palavras: *o agente escolhe a ação; o ambiente determina seu resultado*.

Também podemos escrever uma equação usando apenas valores de ação. Como $V^*(s') = \max_{a'} Q^*(s', a')$, temos:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right].$$

Se s' for terminal, o termo de continuação é zero.

Essa é a equação de Bellman de otimalidade para Q^* . Ela será a utilizada na implementação do algoritmo Q-learning, apresentado mais adiante nesta nota de aula.

3.4.4 Interpretando a estrutura recursiva

As equações de Bellman expressam um princípio de otimalidade recursiva:

Se conhecemos o valor ótimo de cada possível próximo estado, podemos avaliar as ações do estado atual e escolher a melhor.

O problema de maximizar o retorno ao longo de toda a interação é decomposto em decisões locais que incorporam o valor da continuação. Essa estrutura fundamenta métodos de programação dinâmica.

A Figura 4 representa essa decomposição. O estado s aparece à esquerda; o par (s, a) , no centro; e os possíveis sucessores, à direita. A figura mostra uma ação a . Para determinar $V^*(s)$, comparamos os valores de todas as ações disponíveis.

1. Começando pela esquerda, o nó s representa o estado atual do agente. A partir dele, o agente escolhe uma ação a , o que o leva ao q-estado (s, a) , uma situação intermediária em que a ação já foi decidida, mas seus efeitos ainda não se materializaram. Esse q-estado encapsula a pergunta: “qual será o valor de executar a em s ?”
2. A partir de (s, a) , o ambiente assume o controle e determina estocasticamente o próximo estado. Isso é representado pelos múltiplos estados sucessores s'_1, s'_2, s'_3 , cada um associado a uma probabilidade de transição $T(s, a, s'_i)$ e a uma recompensa imediata r_i . Essa ramificação explicita que o resultado de uma ação não é único, mas uma distribuição de possibilidades.

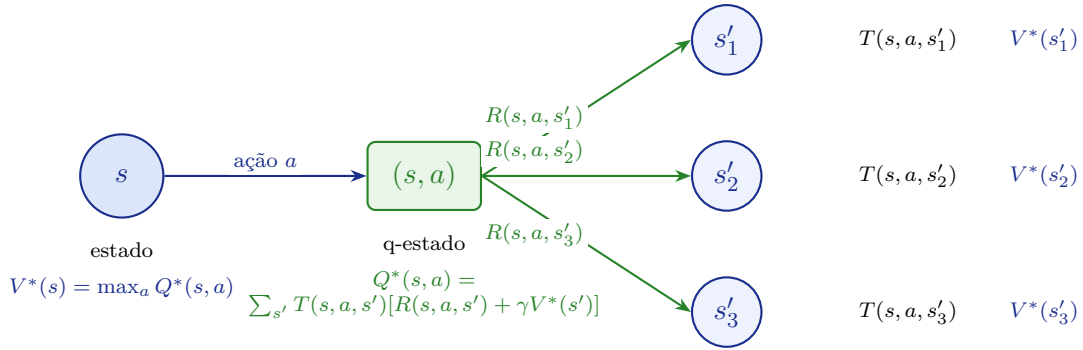


Figura 4: Estrutura das equações de Bellman de otimalidade. Para avaliar uma ação, combinamos recompensa imediata e valor descontado da continuação, calculando uma média ponderada sobre os sucessores. Para avaliar o estado, tomamos o máximo sobre os valores das ações disponíveis.

3. Para cada estado sucessor s'_i , consideramos seu valor ótimo $V^*(s'_i)$, que representa a utilidade esperada acumulada a partir daquele ponto, assumindo comportamento ótimo no futuro. O valor $Q^*(s, a)$ é então calculado como uma média ponderada dessas contribuições: cada termo combina a recompensa imediata r_i com o valor futuro descontado $\gamma V^*(s'_i)$, ponderado pela probabilidade $T(s, a, s'_i)$.
4. Por fim, o valor do estado $V^*(s)$ é obtido selecionando a melhor ação disponível, isto é, tomando o máximo sobre todos os q-estados (s, a) . Assim, a figura explicita a decomposição do problema: o agente escolhe uma ação (controle), o ambiente responde estocasticamente (incerteza), e o valor resulta da combinação entre recompensa imediata e valor esperado do futuro.

3.4.5 Exemplo resolvido no Mundo Grade

As equações anteriores caracterizam os valores ótimos, mas não pressupõem que já os conheçamos. Podemos usar sua estrutura para atualizar estimativas de valor. Essa operação é chamada de *backup de Bellman*: usamos os valores estimados dos sucessores para produzir uma nova avaliação.

Avaliando a ação Norte. Considere o estado $s = (3, 2)$ do Mundo Grade. Ao executar a ação Norte, os possíveis resultados são:

- $(3, 3)$ com probabilidade 0,8;
- $(3, 2)$ com probabilidade 0,1, pois o movimento para Oeste encontra o obstáculo;
- $(4, 2)$ com probabilidade 0,1, entrando no terminal negativo.

Assuma as estimativas:

$$\hat{V}(3, 3) = 0,8, \quad \hat{V}(3, 2) = 0,5,$$

e o fator de desconto $\gamma = 0,9$.

As transições para estados não terminais recebem recompensa $-0,04$. A entrada no terminal negativo recebe recompensa -1 , sem valor de continuação.

O backup para a ação Norte é:

$$\begin{aligned}\hat{Q}(s, \text{Norte}) &= 0,8(-0,04 + 0,9 \cdot 0,8) \\ &\quad + 0,1(-0,04 + 0,9 \cdot 0,5) \\ &\quad + 0,1(-1).\end{aligned}$$

Calculando:

$$\begin{aligned}\hat{Q}(s, \text{Norte}) &= 0,8 \cdot 0,68 + 0,1 \cdot 0,41 - 0,1 \\ &= 0,544 + 0,041 - 0,1 \\ &= \boxed{0,485}.\end{aligned}$$

O resultado combina progresso esperado e risco. A transição para (3,3) contribui positivamente, enquanto a possibilidade de entrar no terminal negativo reduz a avaliação da ação.

O valor 0,485 é uma estimativa produzida a partir de outras estimativas. Não demonstramos que $Q^*((3, 2), \text{Norte}) = 0,485$.

Escolhendo uma ação. Suponha que, para completar a comparação, tenhamos as seguintes estimativas para as quatro ações:

$$\begin{aligned}\hat{Q}(s, \text{Norte}) &= 0,485, & \hat{Q}(s, \text{Leste}) &= -0,2, \\ \hat{Q}(s, \text{Sul}) &= 0,3, & \hat{Q}(s, \text{Oeste}) &= 0,4.\end{aligned}$$

Somente a estimativa de Norte foi calculada acima. As demais são valores ilustrativos fornecidos para esta comparação.

A nova estimativa do valor do estado é:

$$\hat{V}_{\text{novo}}(s) = \max_a \hat{Q}(s, a) = \max(0,485, -0,2, 0,3, 0,4) = 0,485.$$

A ação gulosa em relação a essas estimativas é, portanto:

$$\pi_{\text{gulosa}}(3, 2) = \text{Norte}.$$

Essa escolha é a melhor segundo as estimativas atuais. Sua otimalidade depende da qualidade dessas estimativas.

Observe também que o backup usou o valor anterior $\hat{V}(3, 2) = 0,5$ para avaliar a possibilidade de permanecer no mesmo estado. Depois da comparação entre ações, a nova estimativa passou a ser 0,485. Não há contradição: estamos atualizando uma estimativa, não resolvendo a equação ótima em um único passo.

3.4.6 Das equações ao aprendizado

O exemplo usou as probabilidades de transição e as recompensas do modelo para calcular uma média sobre os sucessores. Quando esse modelo é conhecido, métodos de programação dinâmica podem aplicar backups repetidamente para calcular valores.

No cenário livre de modelo estudado nesta nota, o agente não conhece T nem R explicitamente. Ele observa transições: executa uma ação, recebe uma recompensa e identifica o próximo estado.

Os métodos de Diferença Temporal apresentados a seguir usam essas experiências para atualizar estimativas de valor. No Q-learning, cada transição fornece um alvo amostrado com a estrutura:

$$\text{recompensa observada} + \gamma \cdot \text{maior valor estimado no próximo estado}.$$

Assim, as equações de Bellman fornecem a relação que os valores ótimos devem satisfazer; o algoritmo de aprendizado especifica como ajustar as estimativas a partir da experiência.

4 Aprendizado por Diferença Temporal

Até aqui, formulamos o problema de AR e derivamos os valores V^* e Q^* por meio da equação de Bellman. O passo seguinte é: como um agente *aprende* esses valores sem conhecer T nem R ? Esta seção apresenta o arcabouço de **Diferença Temporal** (TD, de *Temporal Difference*), que é a base de todos os algoritmos de AR baseados em valor estudados a partir de agora, incluindo Q-learning e SARSA.

4.1 Três formas de aprender valores

Há três abordagens fundamentais para estimar valores em AR. Compreendê-las comparativamente é essencial para entender por que o TD é a escolha padrão.

Programação Dinâmica (DP). Se o agente conhece T e R , pode aplicar a equação de Bellman diretamente, iterando sobre todos os estados até a convergência (Iteração de Valor). O problema: em AR livre de modelo, T e R são desconhecidos, de modo que essa abordagem não está disponível.

Monte Carlo (MC). O agente executa episódios completos e usa o retorno real acumulado $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$ como estimativa do valor. A atualização ocorre somente *ao final de cada episódio*:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G_t - Q(s, a)]$$

Para uma política fixa π , G_t é uma estimativa não viesada de $Q^\pi(s, a)$, condicionada ao par atual e seguindo π na continuação; não necessariamente de $Q^*(s, a)$: usa apenas recompensas reais, sem estimativas intermediárias. A desvantagem é que o agente precisa esperar o fim do episódio para atualizar qualquer valor, o que pode ser lento ou impossível em tarefas sem término natural.

Diferença Temporal (TD). O agente atualiza os valores *a cada passo*, sem esperar o fim do episódio, usando uma estimativa do retorno futuro construída a partir dos valores Q já aprendidos. Em vez do retorno real G_t (que requer o episódio completo), o TD usa um **alvo de um passo**:

$$\underbrace{r_t + \gamma \cdot \hat{Q}(s_{t+1}, a')}_{\text{alvo TD}} \quad \text{no lugar de} \quad \underbrace{G_t}_{\text{alvo MC}}$$

Esse mecanismo de usar estimativas para melhorar estimativas é denominado **bootstrap**.

Método	Precisa de T , R ?	Espera fim do episódio?	Usa bootstrap?
Programação Dinâmica	Sim	Não	Sim
Monte Carlo	Não	Sim	Não
Diferença Temporal	Não	Não	Sim

O TD combina o melhor dos dois mundos: dispensa o modelo (como MC) e atualiza a cada passo (como DP). Esse é o motivo pelo qual é o arcabouço dominante em AR livre de modelo.

Imagine um estudante estimando sua nota final durante a prova. Na abordagem Monte Carlo, ele só revisaria sua estimativa após ver o gabarito completo. Na abordagem TD, ele revisaria sua estimativa a cada questão respondida: “essa questão foi mais difícil do que eu esperava; preciso revisar minha estimativa de nota final”. O TD aprende da mesma forma: cada transição (s, a, r, s') é uma nova evidência que permite corrigir a estimativa atual, sem esperar o fim do episódio.

4.2 O erro de Diferença Temporal

O conceito central do arcabouço TD é o **erro de diferença temporal** δ_t , que mede a discrepância entre o valor estimado antes de uma transição e a evidência obtida após ela:

$$\delta_t = \underbrace{r_t + \gamma \hat{V}(s_{t+1})}_{\text{alvo: o que foi observado}} - \underbrace{\hat{V}(s_t)}_{\text{previsão: o que se esperava}}$$

Essa equação tem uma interpretação direta e poderosa:

- $\delta_t > 0$: a transição foi **melhor do que o esperado**. O agente subestimava o valor de s_t ; deve aumentar $\hat{V}(s_t)$.
- $\delta_t < 0$: a transição foi **pior do que o esperado**. O agente superestimava o valor de s_t ; deve diminuir $\hat{V}(s_t)$.
- $\delta_t = 0$: o alvo desta amostra coincide com a previsão. Não há atualização nesta amostra, mas isso não prova que o valor estimado seja correto.

A atualização TD genérica é então:

$$\hat{V}(s_t) \leftarrow \hat{V}(s_t) + \alpha \cdot \delta_t$$

Como o alvo TD usa $\hat{V}(s_{t+1})$, que é uma estimativa, não o valor verdadeiro, as atualizações TD introduzem um **viés**: se os valores atuais estiverem distantes do valor da política avaliada, o alvo pode apontar na direção errada. Em contrapartida, MC não tem esse viés, pois usa recompensas reais. A desvantagem de MC é a alta variância: o retorno G_t depende de toda a sequência de recompensas do episódio, que pode ser ruidosa. O TD troca algum viés por menor variância e atualização imediata. Na prática, o TD converge mais rápido que MC na maioria dos problemas, apesar do viés inicial.

4.3 TD e a equação de Bellman

A conexão entre TD e a equação de Bellman é direta e fundamental. Recorde a equação de Bellman para Q^* :

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

No cenário livre de modelo estudado, T é desconhecida. A ideia do TD é usar uma amostra (r, s') para estimar um backup de Bellman das estimativas atuais. Para o alvo do Q-learning, temos:

$$\begin{aligned} \mathbb{E} \left[r + \gamma \max_{a'} Q(s', a') \mid s, a, Q \right] \\ = \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \right]. \end{aligned}$$

Aqui condicionamos à estimativa atual Q , e $R(s, a, s')$ representa a recompensa esperada da transição. No terminal, o valor de continuação é zero. O alvo amostrado estima o backup de Bellman de Q , que ainda pode estar errado; coincide com a expressão contendo V^* quando $\max_{a'} Q(s', a') = V^*(s')$ nos sucessores relevantes. A convergência do aprendizado exige condições adicionais, apresentadas adiante.

Essa é a ideia-chave de todos os algoritmos TD: **aprender, a partir de amostras, o mesmo tipo de valor que a equação de Bellman definiria se T fosse conhecida**.

Q-learning e SARSA, apresentados na próxima seção, são ambos algoritmos TD aplicados a q-valores. A única diferença entre eles está em *como* constroem o alvo TD: qual estimativa de valor futuro usam. Essa diferença aparentemente pequena tem consequências práticas importantes, que ficam claras quando se entende que um algoritmo é *off-policy* e o outro é *on-policy*. Assim, os três métodos diferem pelo tipo de informação disponível e pelo momento da atualização: a programação dinâmica usa o modelo, Monte Carlo espera o retorno completo do episódio e não faz *bootstrap*, e TD atualiza a cada transição usando estimativas atuais como parte do alvo. Essa última propriedade é justamente o *bootstrap*.

5 Métodos Tabulares de Diferença Temporal

5.1 Q-learning

5.1.1 Ideia central

O **Q-learning** é o algoritmo TD livre de modelo mais clássico do AR. Seu objetivo é aprender $Q^*(s, a)$ diretamente a partir da experiência, sem nunca conhecer T ou R explicitamente.

A ideia é simples: o agente mantém uma tabela com uma entrada $Q(s, a)$ para cada par estado-ação. No início, todas as entradas são zero. A cada transição, isto é, a cada vez que o agente executa uma ação, observa a recompensa e o próximo estado, ele atualiza a entrada correspondente na direção correta, usando a equação de Bellman como referência.

Sob as condições de convergência tabular estabelecidas adiante, os valores na tabela convergem para os valores ótimos $Q^*(s, a)$. No limite, uma política ótima é obtida por $\pi^*(s) \in \arg \max_a Q^*(s, a)$. Durante o aprendizado, o máximo de Q define uma política gulosa em relação às estimativas atuais.

Na seção anterior, aplicamos explicitamente a equação de Bellman no Mundo Grade para calcular o valor de uma ação, combinando probabilidades de transição, recompensas e valores futuros.

A equação de Bellman define $Q^*(s, a)$ exatamente como:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Mas no cenário desta nota há um problema fundamental: **não conhecemos** T , portanto não podemos calcular essa soma diretamente.

A solução do Q-learning é elegante: em vez de calcular a *média exata* sobre todos os s' possíveis (o que exigiria T), o agente usa a *amostra observada* (i.e., o estado s' que realmente ocorreu na transição) como um estimador dessa média:

$$y = r + \gamma \max_{a'} Q(s', a')$$

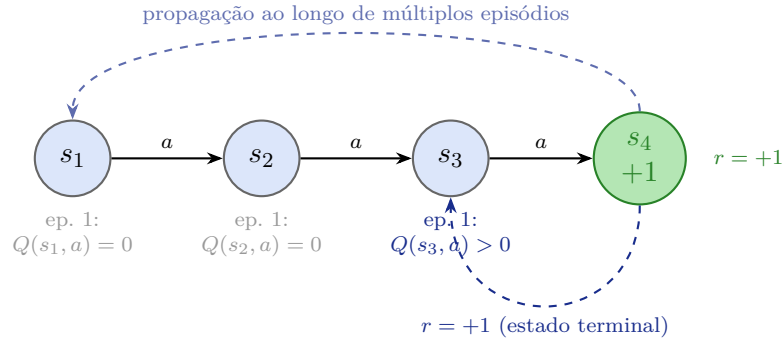
Esse alvo é uma amostra do backup de Bellman das estimativas atuais, não do valor ótimo já conhecido. No terminal, o alvo é apenas r . Sob as condições de convergência tabular estabelecidas adiante, as atualizações sucessivas levam Q ao ponto fixo Q^* .

Em outras palavras, o Q-learning aprende automaticamente, a partir de amostras, o mesmo tipo de valor que calculamos explicitamente no exemplo do Mundo Grade. A Figura 5 ilustra um aspecto central do Q-learning: os valores Q não surgem de uma única vez, mas são aprendidos de forma incremental e propagam-se gradualmente a partir dos estados terminais em direção aos estados iniciais.

- Considere a cadeia de estados $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$, onde s_4 é terminal com recompensa $+1$. No primeiro episódio, o agente percorre essa sequência e observa a recompensa apenas ao atingir o estado terminal. Como consequência, apenas o valor de $Q(s_3, a)$ é diretamente atualizado, pois o alvo de Bellman depende de $r + \gamma \max_{a'} Q(s_4, a')$, e $Q(s_4, \cdot) = 0$. Assim, apenas o estado imediatamente anterior ao terminal recebe informação útil nesse primeiro momento.
- Nos episódios seguintes, o valor aprendido em s_3 passa a influenciar estados anteriores. Quando o agente visita s_2 , o alvo de atualização passa a incluir o termo $\gamma \max_{a'} Q(s_3, a')$, que agora já é positivo. Isso faz com que $Q(s_2, a)$ aumente. O mesmo ocorre posteriormente com s_1 , à medida que episódios adicionais são executados.

Esse processo corresponde exatamente à aplicação iterativa da equação de Bellman em sua forma amostrada: cada atualização incorpora uma estimativa do valor futuro, permitindo que a informação sobre recompensas distantes seja transmitida gradualmente ao longo da cadeia de estados. Dessa forma, o aprendizado no Q-learning pode ser interpretado como um processo de **propagação de valor**: recompensas observadas em estados terminais são retropropagadas, passo a passo, para estados predecessores, ao longo de múltiplos episódios. Isso explica por que estados mais distantes do terminal exigem mais interações para convergir.

O Q-learning é “livre de modelo” porque nunca constrói explicitamente uma estimativa de $T(s, a, s')$ ou $R(s, a, s')$. Em vez disso, aprende Q^* diretamente a partir das transições observadas. Isso o distingue de métodos “baseados em modelo”, que primeiro estimam



Observação: a cada episódio, o valor se propaga um passo a mais em direção ao estado inicial. Após k episódios ($k = 1, 2, 3$), $Q(s_{4-k}, a)$ já foi atualizado.

Figura 5: Propagação dos valores Q ao longo dos episódios no Q-learning tabular. No primeiro episódio, apenas o Q-valor do estado imediatamente anterior ao terminal (s_3) é atualizado. Nos episódios seguintes, o valor se propaga gradualmente para s_2 e s_1 via o termo $\gamma \cdot \max_{a'} Q(s', a')$ aplicado aos estados não terminais.

T e R e depois resolvem o PDM resultante. O processo ilustrado na Figura 5 ocorre diretamente no Mundo Grade estudado anteriormente.

Considere um episódio típico em que o agente parte de $(1, 1)$ e alcança o estado terminal positivo $(4, 3)$. No primeiro episódio, apenas o estado imediatamente anterior ao terminal, por exemplo, $(3, 3)$, terá seu valor Q atualizado de forma significativa, pois é o único que observa diretamente a recompensa $+1$.

Nos episódios seguintes, esse valor começa a influenciar estados anteriores no caminho. Quando o agente passa por $(2, 3)$, a atualização de $Q((2, 3), a)$ passa a incorporar o termo $\gamma \max_{a'} Q((3, 3), a')$, que agora já é positivo. Esse efeito continua ao longo dos episódios, atingindo gradualmente estados mais distantes, como $(1, 3)$, $(1, 2)$ e, eventualmente, $(1, 1)$.

Esse comportamento explica um fenômeno importante: estados próximos ao objetivo aprendem rapidamente, enquanto estados distantes demoram mais para refletir o valor da recompensa final. A informação precisa ser propagada ao longo das trajetórias observadas, o que exige múltiplas visitas e episódios.

Assim, o Q-learning não “descobre” imediatamente a melhor política. Ele a constrói progressivamente, à medida que a informação sobre recompensas se difunde pelo espaço de estados.

5.1.2 Regra de atualização

A cada transição (s, a, r, s') , o Q-learning atualiza $Q(s, a)$ de acordo com a seguinte regra:

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a')]$$

onde $\alpha \in (0, 1]$ é a taxa de aprendizado e $\gamma \in [0, 1]$ é o fator de desconto.

A expressão entre colchetes é o alvo (*target*): representa o que o agente aprendeu com a transição atual. A diferença entre o alvo e o valor atual é o erro de diferença temporal δ :

$$\delta = [r + \gamma \cdot \max_{a'} Q(s', a')] - Q(s, a)$$

A regra de atualização pode então ser reescrita de forma equivalente como:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \delta$$

Lida dessa forma, a atualização diz: *ajuste o valor atual de $Q(s, a)$ proporcionalmente ao erro entre o que você esperava e o que observou*. Quando $\delta > 0$, o resultado foi melhor do que o esperado e $Q(s, a)$ sobe. Quando $\delta < 0$, o resultado foi pior do que o esperado e $Q(s, a)$ desce. Quando $\delta = 0$, o alvo desta amostra coincide com a previsão e não há atualização; isso não prova que a estimativa seja correta em geral.

A presença de $\max_{a'}$ no alvo é o elemento que torna o Q-learning *off-policy*: ele pode aprender Q^* mesmo que a política de exploração usada durante o treinamento não seja a política ótima.

A lógica é: independentemente de qual ação o agente *realmente* vai executar em s' , o alvo usa uma continuação gulosa em relação às estimativas atuais de Q . Durante o aprendizado, a ação de maior Q pode ainda não ser ótima. A operação de máximo corresponde ao operador de Bellman de otimalidade, cujo ponto fixo é Q^* .

Para comparação, um algoritmo *on-policy* usaria $Q(s', a_{\text{exploração}})$, que é o valor da ação que o agente realmente escolheu em s' . Esse alvo é coerente com a avaliação da política seguida, incluindo sua exploração; não é uma contaminação. O Q-learning e o SARSA diferem na política cuja continuação é avaliada: o primeiro usa o maior valor estimado, enquanto o segundo usa a ação selecionada pela política de comportamento. O SARSA, apresentado na próxima seção, é exatamente esse algoritmo *on-policy* e usa a ação realmente escolhida. O operador $\max_{a'}$ no alvo explica a propriedade *off-policy*: a atualização avalia a ação de maior valor estimado em s' , independentemente da ação que a política de exploração realmente executará.

5.1.3 Exemplo resolvido

Este exemplo ilustra um passo de aplicação da regra de atualização. Considere o Mundo Grade com $\gamma = 0,9$ e $\alpha = 0,5$. O agente está em $s = (1, 1)$, executa Norte, transita para $s' = (1, 2)$ com $r = -0,04$. Os valores atuais são $Q((1, 1), \text{Norte}) = 0$ e $\max_{a'} Q((1, 2), a') = 0,50$ (suponha que o agente já aprendeu algo sobre $(1, 2)$).

Passo 1: calcular o alvo.

$$r + \gamma \cdot \max_{a'} Q(s', a') = -0,04 + 0,9 \times 0,50 = -0,04 + 0,45 = 0,41$$

Passo 2: calcular o erro temporal.

$$\delta = 0,41 - 0 = 0,41$$

Passo 3: atualizar.

$$Q((1, 1), \text{Norte}) \leftarrow 0 + 0,5 \times 0,41 = \mathbf{0,205}$$

O valor de $Q((1, 1), \text{Norte})$ subiu de 0 para 0,205, refletindo que ir para Norte a partir de $(1, 1)$ parece promissor: afinal, $(1, 2)$ já tem um valor razoável aprendido.

5.1.4 Taxa de aprendizado

A **taxa de aprendizado** $\alpha \in (0, 1]$ controla o quanto cada nova experiência altera o valor atual de $Q(s, a)$:

- $\alpha = 1$: o valor $Q(s, a)$ é substituído completamente pelo alvo da transição mais recente. O agente ignora todo o histórico passado e “confia cegamente” na transição atual. Isso é problemático em ambientes estocásticos, onde uma única transição pode ser atípica.
- α próximo de 0: o valor $Q(s, a)$ quase não é alterado. O agente aprende muito lentamente.
- α intermediário (e.g., 0,1 a 0,5): o agente faz uma média ponderada entre o que já sabia e o que aprendeu na transição atual, equilibrando estabilidade e velocidade de aprendizado.

Na prática, α pode ser fixo ou decrescente ao longo do treinamento. Uma estratégia comum é diminuir α gradualmente: no início (quando há pouco conhecimento), o agente deve aprender rapidamente; no final (quando o conhecimento é mais maduro), ele deve atualizar com mais cautela.

Existem condições matemáticas precisas sobre α para que o Q-learning convirja para Q^* . Para cada par estado-ação, a sequência de taxas deve satisfazer as condições de Robbins–Monro: $\sum_t \alpha_t = \infty$ e $\sum_t \alpha_t^2 < \infty$. Além disso, cada par deve ser visitado infinitas vezes. Uma taxa fixa pequena, como $\alpha = 0,1$, é comum em aplicações, mas não satisfaz a segunda condição e, em geral, produz estimativas que continuam oscilando em torno dos valores aprendidos, em vez de convergirem exatamente.

5.1.5 Convergência e propriedades

O Q-learning possui uma propriedade notável: sob condições adequadas, ele converge para os valores ótimos Q^* *independentemente* da política usada para explorar o ambiente. No enquadramento tabular descontado usual, as condições incluem:

- Espaços de estados e ações finitos, ambiente estacionário, recompensas limitadas e $0 \leq \gamma < 1$.
- Cada par não terminal (s, a) deve ser visitado infinitas vezes durante o treinamento.
- A taxa de aprendizado deve satisfazer $0 < \alpha_t(s, a) \leq 1$ e as condições de Robbins–Monro ao longo das atualizações de cada par.

A prova de convergência foi estabelecida em 1992 por Watkins e Dayan¹. O Q-learning é, portanto, um algoritmo com garantia teórica, sob essas hipóteses, com convergência quase certa (probabilidade 1). Visitar um par algumas vezes não torna seu valor exato. O pseudocódigo a seguir resume o algoritmo completo:

¹[Watkins and Dayan, 1992]

5.1.6 Pseudocódigo

O Algoritmo 1 resume o funcionamento completo do Q-learning tabular ao longo de múltiplos episódios de interação com o ambiente. Inicialmente, todos os q-valores $Q(s, a)$ são zerados, refletindo a ausência de conhecimento prévio do agente. Em cada episódio, o agente parte de um estado inicial e repete o mesmo ciclo até alcançar um estado terminal: escolhe uma ação segundo uma política de exploração, executa essa ação, observa a recompensa recebida e o próximo estado, e então atualiza a entrada correspondente $Q(s, a)$. O ponto central do algoritmo está na regra de atualização, que ajusta o valor atual na direção do alvo $r + \gamma \max_{a'} Q(s', a')$. Esse alvo combina a recompensa imediata observada com a melhor estimativa disponível para o futuro a partir de s' , o que faz o algoritmo ser *off-policy*: embora o agente explore durante o treinamento, a atualização usa uma continuação gulosa em relação às estimativas atuais. Se s' for terminal, o alvo é apenas r . Ao final de muitos episódios, essa sequência de interações e atualizações faz com que os valores Q se aproximem de Q^* , permitindo derivar a política gulosa correspondente.

Algoritmo 1 Q-Learning (*off-policy*)

```

1: entrada: taxa de aprendizado  $\alpha$ , fator de desconto  $\gamma$ , número de episódios
2: Inicializar  $Q(s, a) \leftarrow 0$  para todo par não terminal  $(s, a)$ 
3: for cada episódio do
4:    $s \leftarrow$  estado inicial
5:   while  $s$  não é terminal do
6:      $a \leftarrow$  escolher ação em  $s$  segundo a política de exploração
7:     executar  $a$ ; observar recompensa  $r$  e próximo estado  $s'$ 
8:     if  $s'$  é terminal then
9:        $y \leftarrow r$ 
10:    else
11:       $y \leftarrow r + \gamma \max_{a'} Q(s', a')$ 
12:    end if
13:     $Q(s, a) \leftarrow Q(s, a) + \alpha [y - Q(s, a)]$ 
14:     $s \leftarrow s'$ 
15:  end while
16: end for
17: return  $Q$ 

```

5.2 SARSA

O SARSA é outro algoritmo TD para aprender q-valores, intimamente relacionado ao Q-learning. O nome é um acrônimo da quintupla que define cada atualização realizada pelo algoritmo: $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$.

5.2.1 Regra de atualização

A cada transição, o agente observa (s, a, r, s') , se s' não for terminal, escolhe a próxima ação a' segundo sua política de exploração *antes* de atualizar, e então aplica:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot Q(s', a') - Q(s, a)]$$

onde a' é a ação *realmente escolhida* no próximo estado s' pela política de exploração. Se s' for terminal, não há próxima ação e o alvo é apenas r .

Para transições não terminais, a diferença em relação ao Q-learning está na continuação: o alvo usa $Q(s', a')$ (o valor da ação que o agente *de fato vai executar*), em vez de $\max_{a'} Q(s', a')$ (o valor da *melhor* ação disponível). A Tabela 1 compara os dois algoritmos diretamente.

Aspecto	Q-learning	SARSA
Alvo de atualização	$r + \gamma \cdot \max_{a'} Q(s', a')$	$r + \gamma \cdot Q(s', a')$, com $a' \sim \pi_{\text{exploração}}$
Tipo de política	<i>Off-policy</i>	<i>On-policy</i>
Quintupla usada	(s, a, r, s')	(s, a, r, s', a')
O que aprende	Busca Q^* com continuação gulosa nas estimativas	Avalia a continuação da política seguida
Perto de regiões perigosas	O alvo não inclui futuras ações exploratórias	O alvo inclui ações da política exploratória

Tabela 1: Comparação entre Q-learning e SARSA. A única diferença na regra de atualização é o alvo.

Os alvos das tabelas são para transições não terminais; em ambos os algoritmos, o alvo terminal é r . Para uma política fixa π , o SARSA tabular avalia Q^π sob as condições usuais de visitas e taxas de aprendizado. No controle, a política muda junto com Q . Sua convergência para Q^* exige, além das condições tabulares, uma política **GLIE** (*greedy in the limit with infinite exploration*): tornar-se guloso no limite, preservando visitas infinitas a todos os pares não terminais. Apenas $\varepsilon \rightarrow 0$ não garante essa exploração.

5.2.2 On-policy versus off-policy: intuição

A distinção *on-policy/off-policy* tem uma consequência prática fundamental, especialmente visível no Mundo Grade com exploração ε -greedy e o precipício em (4, 2).

- O Q-learning aprende Q^* : os valores que o agente teria se ele sempre agisse de forma ótima. Ao usar $\max_{a'}$ no alvo, ele *ignora* o fato de que, durante o treinamento, o agente também executa ações aleatórias com probabilidade ε . A política aprendida é a ótima para um agente sem exploração.
- O SARSA aprende o valor da política *que está sendo de fato executada*, incluindo as ações aleatórias da exploração. Isso faz com que ele perceba que estados próximos ao precipício são perigosos não apenas em teoria, mas na prática do treinamento. Como resultado, o SARSA tende a aprender caminhos *mais afastados* de zonas de risco.

A Figura 6 ilustra essa diferença no ambiente *Cliff Walking*², um exemplo clássico em que a distinção entre comportamento *off-policy* e *on-policy* aparece com clareza. Nesse ambiente, o agente parte do estado inicial S , precisa alcançar o objetivo G , e as células do penhasco (*cliff*) produzem uma penalidade severa quando visitadas. O Q-learning tende a aprender a trajetória mais curta até o objetivo, passando rente ao penhasco, porque sua atualização assume que, no futuro, o agente sempre escolherá a melhor ação disponível. Já o SARSA aprende o valor da política ε -greedy que está sendo realmente executada durante o treinamento; por isso, ele leva em conta o risco de que uma ação exploratória faça o agente cair no penhasco. Em consequência, o SARSA tende a preferir trajetórias mais longas, porém mais seguras.

A figura mostra, portanto, um ponto conceitual central: o Q-learning aprende a política ótima para um agente que agirá de forma gulosa no futuro, enquanto o SARSA aprende o valor da política exploratória que está sendo realmente seguida. Em ambientes com regiões muito perigosas, como o *Cliff Walking*, essa diferença faz o Q-learning tender a trajetórias mais curtas e arriscadas, enquanto o SARSA tende a trajetórias mais longas, porém mais seguras.

A escolha entre os dois algoritmos depende do contexto:

- **Q-learning** é preferível quando o objetivo é aprender a política mais eficiente possível e o agente pode “treinar” separado de “operar”, por exemplo, em simulação. O risco durante a fase de exploração não tem custo real.
- **SARSA** é preferível quando o agente treina e opera ao mesmo tempo (*aprendizado online*), especialmente em domínios com estados perigosos (robótica, sistemas de controle). O algoritmo naturalmente inclui o custo das explorações na política aprendida.

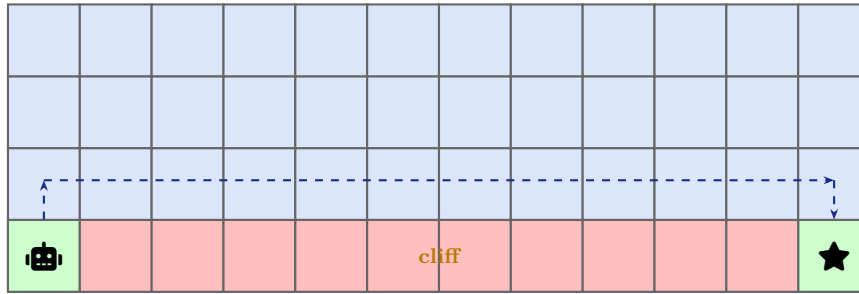
5.2.3 Pseudocódigo

O Algoritmo 2 resume o funcionamento completo do SARSA ao longo de múltiplos episódios de interação com o ambiente. Sua estrutura geral é muito próxima à do Q-learning apresentado anteriormente: o agente inicializa a tabela Q , percorre episódios sucessivos, escolhe ações, observa recompensas e atualiza estimativas de valor. A diferença crucial aparece no fato de que, no SARSA, a próxima ação a' precisa ser escolhida antes da atualização, pois o alvo usa o valor $Q(s', a')$ da ação que será efetivamente executada no próximo estado. Por isso, o algoritmo incorpora diretamente a política de exploração em seu processo de aprendizado, caracterizando seu comportamento *on-policy*. A escolha de a' e sua utilização no alvo evidenciam esse ponto: o SARSA não aprende assumindo que o agente agirá otimamente no próximo passo, mas sim assumindo que continuará agindo segundo a política que está de fato sendo seguida durante o treinamento.

²*Cliff Walking* é um ambiente clássico de aprendizado por reforço, popularizado por Sutton e Barto [Sutton and Barto, 2018], no qual o agente deve ir de um estado inicial S até um objetivo G atravessando uma grade. As células entre S e G , na borda inferior, formam um “penhasco” (*cliff*): se o agente entra em uma delas, recebe uma penalidade severa e retorna ao estado inicial. O exemplo é usado para ilustrar a diferença entre políticas mais curtas porém arriscadas e políticas mais longas porém mais seguras.

Q-learning (*off-policy*)

trajetória mais curta e mais arriscada



SARSA (*on-policy*)

trajetória mais longa e mais segura

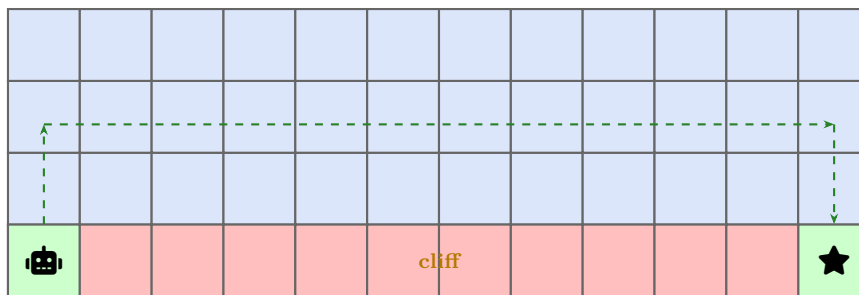


Figura 6: Diferença entre Q-learning e SARSA no ambiente *Cliff Walking*. O Q-learning aprende a política ótima para um agente que age de forma gulosa no futuro e, por isso, tende a escolher o caminho mais curto, rente ao penhasco. O SARSA aprende o valor da política exploratória realmente executada durante o treinamento e, por levar em conta o risco das ações aleatórias, tende a preferir um caminho mais seguro e afastado da borda perigosa.

Algoritmo 2 SARSA (*on-policy*)

```

1: entrada: taxa de aprendizado  $\alpha$ , fator de desconto  $\gamma$ , número de episódios
2: Inicializar  $Q(s, a) \leftarrow 0$  para todo par não terminal  $(s, a)$ 
3: for cada episódio do
4:    $s \leftarrow$  estado inicial
5:   if  $s$  não é terminal then
6:      $a \leftarrow$  escolher ação em  $s$  segundo  $\varepsilon$ -greedy
7:   end if
8:   while  $s$  não é terminal do
9:     executar  $a$ ; observar recompensa  $r$  e próximo estado  $s'$ 
10:    if  $s'$  é terminal then
11:       $y \leftarrow r$ 
12:    else
13:       $a' \leftarrow$  escolher ação em  $s'$  segundo  $\varepsilon$ -greedy
14:       $y \leftarrow r + \gamma Q(s', a')$ 
15:    end if
16:     $Q(s, a) \leftarrow Q(s, a) + \alpha [y - Q(s, a)]$ 
17:     $s \leftarrow s'$ 
18:    if  $s$  não é terminal then
19:       $a \leftarrow a'$ 
20:    end if
21:  end while
22: end for

```

No pseudocódigo do SARSA, a ação inicial é escolhida segundo a política de exploração; em cada transição não terminal, a próxima ação a' também é escolhida antes da atualização, e o alvo usa $Q(s', a')$. Na transição terminal, o alvo é r e nenhuma próxima ação é escolhida. No Q-learning, por contraste, o alvo usa o máximo sobre as ações e não depende da ação que será executada em s' .

Pergunta 5

O Q-learning é descrito como um algoritmo *off-policy*. Isso significa que ele pode aprender Q^* mesmo quando a política de exploração usada durante o treinamento não é a política ótima. Analise a regra de atualização e identifique o elemento que garante essa propriedade. Em seguida, contraste com o SARSA: que diferença faz usar $Q(s', a_{\text{exploração}})$ em vez de $\max_{a'} Q(s', a')$? Em que tipo de ambiente essa diferença seria mais relevante?

5.3 Exemplo resolvido: comparação entre Q-learning e SARSA

Utilizamos o mesmo estado do exemplo anterior para ilustrar concretamente a diferença entre os dois algoritmos.

Contexto : agente em $s = (1, 1)$, executa Norte, chega a $s' = (1, 2)$ com $r = -0,04$. Parâmetros: $\gamma = 0,9$, $\alpha = 0,5$, $Q((1, 1), \text{Norte}) = 0$.

A política ε -greedy escolheu, em $s' = (1, 2)$, a ação $a' = \text{Leste}$ (ação exploratória aleatória, não a gulosa nas estimativas atuais). Os valores relevantes são:

$$\max_{a'} Q((1, 2), a') = 0,50 \quad (\text{melhor ação em } s')$$

$$Q((1, 2), \text{Leste}) = 0,10 \quad (\text{ação realmente escolhida em } s')$$

Q-learning (off-policy):

$$\delta_{\text{QL}} = [-0,04 + 0,9 \times 0,50] - 0 = 0,41$$

$$Q_{\text{QL}}((1, 1), \text{Norte}) \leftarrow 0 + 0,5 \times 0,41 = \mathbf{0,205}$$

SARSA (on-policy):

$$\delta_{\text{SARSA}} = [-0,04 + 0,9 \times 0,10] - 0 = 0,05$$

$$Q_{\text{SARSA}}((1, 1), \text{Norte}) \leftarrow 0 + 0,5 \times 0,05 = \mathbf{0,025}$$

Interpretação: o Q-learning atribuiu um valor quase 8 vezes maior para a ação Norte a partir de $(1, 1)$, pois usa uma continuação gulosa em relação às estimativas atuais em $(1, 2)$ (escolhendo Norte, com $Q = 0,50$). O SARSA usou o valor da ação que *foi de fato escolhida* (Leste, com $Q = 0,10$), resultando em uma estimativa mais conservadora. Ao longo de muitos episódios, essa diferença faz com que o SARSA aprenda a valorizar menos regiões que o agente visitará frequentemente por acidente durante a exploração.

6 Exploração versus Aproveitamento

6.1 O dilema fundamental

Durante o treinamento, o agente enfrenta um dilema fundamental: para aprender a política ótima, ele precisa **explorar** o espaço de estados, isto é, visitar estados e ações que ainda não experimentou, para descobrir se são bons ou ruins. Mas para maximizar as recompensas durante o treinamento, ele deve **aproveitar** o que já aprendeu, executando as ações que, até o momento, parecem as melhores.

Essa tensão é estrutural:

- Um agente puramente **explorativo** (que sempre escolhe ações aleatórias) eventualmente visita todos os pares (s, a) , mas acumula recompensas baixas durante o processo.
- Um agente puramente **aproveitador** (que sempre escolhe a ação de maior Q atual) pode nunca descobrir ações melhores em regiões do espaço de estados não visitadas, ficando preso em uma política subótima.

A melhor estratégia de longo prazo frequentemente exige sacrifícios de curto prazo: o agente deve arriscar ações desconhecidas para descobrir se elas são melhores do que as conhecidas.

Um equívoco comum é pensar que explorar significa “agir aleatoriamente”. Na verdade, o objetivo da exploração é mais preciso: **reduzir a incerteza sobre o valor das ações em regiões pouco conhecidas do espaço de estados**.

Agir aleatoriamente é uma forma simples de exploração, mas não a mais eficiente: o agente pode continuar visitando aleatoriamente estados que já conhece bem, desperdiçando oportunidades de explorar regiões novas. Estratégias mais sofisticadas (como funções de exploração ou UCB) direcionam a exploração para onde ela é mais útil: ações com alta incerteza, não apenas ações aleatórias. Sem exploração, uma política que sempre escolhe a ação atualmente estimada como melhor pode deixar de visitar pares estado-ação importantes e permanecer subótima. Para as garantias tabulares de convergência, cada par estado-ação relevante precisa ser visitado repetidamente, além de a taxa de aprendizado satisfazer as condições apropriadas.

A necessidade de exploração pode ser entendida diretamente à luz do processo de propagação de valores discutido anteriormente.

No Q-learning, a informação sobre recompensas é propagada a partir dos estados terminais para estados anteriores, ao longo das trajetórias observadas. No entanto, essa propagação só ocorre se o agente *visitar* esses estados e executar as ações correspondentes.

Considere o Mundo Grade: se o agente nunca experimentar um caminho que leve ao estado $(4, 3)$, ele nunca observará a recompensa $+1$. Como consequência, os valores $Q(s, a)$ permanecerão próximos de zero, e nenhuma informação útil será propagada para os estados anteriores.

De forma análoga, se o agente evitar sistematicamente certas ações (por exemplo, por considerá-las inicialmente pouco promissoras), ele pode deixar de descobrir trajetórias melhores que passam por essas ações.

Assim, a exploração desempenha um papel essencial: ela garante que o agente obtenha amostras suficientes do ambiente para iniciar e sustentar o processo de propagação de va-

lores. Sem exploração, o aprendizado pode estagnar em políticas subótimas simplesmente porque partes relevantes do espaço de estados nunca foram visitadas.

Em resumo: a propagação de valor depende da experiência, e a exploração é o mecanismo que garante essa experiência.

6.2 Estratégia ϵ -greedy

A estratégia mais simples e amplamente usada é a ϵ -greedy, cujo funcionamento está esquematizado na Figura 7:

- Com probabilidade ϵ (pequena), o agente escolhe uma ação *aleatória* entre todas as disponíveis no estado atual (exploração).
- Com probabilidade $1 - \epsilon$ (grande), o agente escolhe a ação que maximiza $Q(s, a)$ no estado atual (aproveitamento).

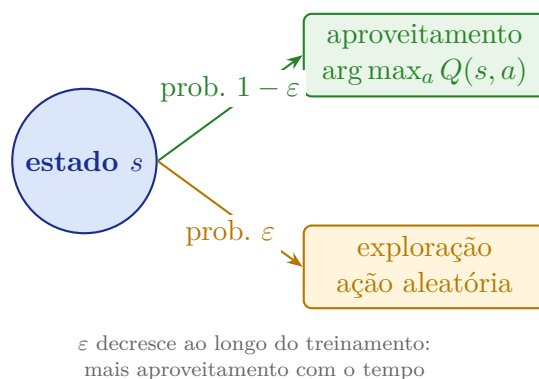


Figura 7: Estratégia ϵ -greedy. A cada passo de tempo, o agente no estado s decide entre *aproveitar* o conhecimento atual (com probabilidade $1 - \epsilon$) e *explorar* o desconhecido (com probabilidade ϵ). Uma variante comum é diminuir ϵ ao longo do treinamento.

Uma melhoria frequente é usar ϵ decrescente: iniciar com ϵ alto (muita exploração, pois o agente sabe pouco) e diminuir progressivamente ao longo do treinamento (menos exploração, pois o agente já aprendeu bastante). Isso garante exploração adequada no início e aproveitamento eficiente no final.

Com ϵ fixo, mesmo após o agente ter aprendido uma boa política, ele continua executando ações aleatórias com probabilidade ϵ . Isso significa que ele não para de explorar, inclusive em regiões do espaço de estados que já foram bem exploradas e cujos valores são confiáveis, acumulando arrependimento (*regret*) por ações evitáveis. Uma programação que reduz ϵ ou uma política gulosa durante a avaliação pode diminuir esse efeito; por isso, é importante distinguir o desempenho durante o treinamento do desempenho em teste.

6.2.1 Exemplo resolvido

Este exemplo ilustra a seleção de ação com ϵ -greedy. Considere um agente no Mundo Grade em $s = (1, 1)$ com $\epsilon = 0,1$ e os seguintes Q-valores aprendidos até o momento:

Estado	Norte	Sul	Leste	Oeste
(1, 1)	0,41	-0,12	0,28	-0,04

Passo 1: determinar a ação de maneira gulosa. A ação de maior Q-valor é Norte (0,41), portanto:

$$a^* = \arg \max_a Q((1, 1), a) = \text{Norte}$$

Passo 2: sortear um número aleatório $x \sim \text{Uniforme}(0, 1)$.

- Se $x > \varepsilon = 0,1$ (probabilidade 0,9): o agente aproveita e executa Norte.
- Se $x \leq \varepsilon = 0,1$ (probabilidade 0,1): o agente explora e sorteia uniformemente entre as 4 ações, obtendo cada uma com probabilidade $0,1/4 = 0,025$.

Passo 3: calcular a probabilidade efetiva de cada ação. Combinando os dois casos:

$$\Pr(\text{Norte}) = 0,9 + 0,025 = 0,925$$

$$\Pr(\text{Sul}) = \Pr(\text{Leste}) = \Pr(\text{Oeste}) = 0,025$$

Interpretação: mesmo após o agente ter aprendido que Norte é a melhor ação em (1, 1), ele ainda executará Sul, Leste ou Oeste com probabilidade total de 0,075 por passo. Após 10.000 passos neste estado, espera-se que ações subótimas sejam escolhidas cerca de 750 vezes, um *regret* acumulado não desprezível mesmo com bom conhecimento adquirido.

6.3 Estratégia softmax (Boltzmann)

Uma alternativa à estratégia ε -greedy é a **exploração softmax**, também conhecida como **exploração de Boltzmann**. Nessa abordagem, o agente seleciona ações de forma probabilística, com probabilidades proporcionais aos seus valores $Q(s, a)$.

Mais precisamente, a probabilidade de escolher a ação a no estado s é dada por:

$$\Pr(a \mid s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}$$

onde $\tau > 0$ é o *parâmetro de temperatura*, que controla o nível de exploração.

- Para τ pequeno, a distribuição se concentra na melhor ação (comportamento quase guloso).
- Para τ grande, as probabilidades se tornam mais uniformes (exploração intensa).

Diferentemente da estratégia ε -greedy, a softmax não trata todas as ações subótimas de forma igual. Ações com valores Q mais altos recebem maior probabilidade de serem escolhidas, enquanto ações claramente ruins são selecionadas com probabilidade muito baixa.

Uma prática comum é usar uma temperatura decrescente: iniciar com τ alto (exploração intensa) e reduzi-lo gradualmente ao longo do treinamento, tornando o comportamento progressivamente mais próximo do aproveitamento puro.

Comparação com ε -greedy. Na estratégia ε -greedy, todas as ações não ótimas são tratadas igualmente durante a exploração. Já na estratégia softmax, a exploração é **informada**: ações melhores são preferidas mesmo durante a fase exploratória. Isso pode levar a um aprendizado mais eficiente, especialmente quando há várias ações razoáveis e poucas claramente ruins.

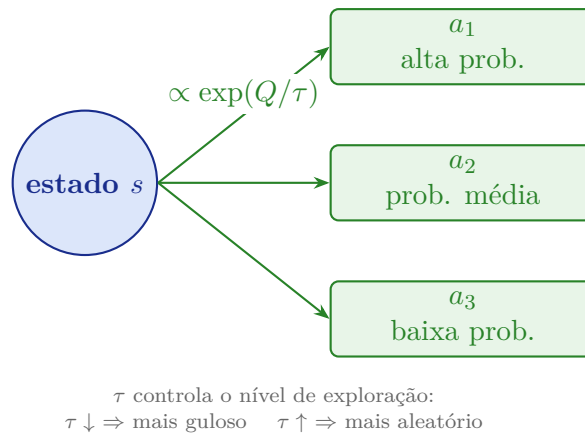


Figura 8: Estratégia softmax. A probabilidade de selecionar cada ação depende suavemente do seu valor $Q(s, a)$: ações melhores são mais prováveis, mas ações subótimas ainda podem ser escolhidas. O parâmetro de temperatura τ controla o equilíbrio entre exploração e aproveitamento.

6.3.1 Exemplo resolvido

Considere o mesmo estado do exemplo anterior:

Estado	Norte	Sul	Leste	Oeste
(1, 1)	0,41	-0,12	0,28	-0,04

Passo 1: calcular os pesos exponenciais. Para $\tau = 0,1$:

$$\exp(0,41/0,1) \gg \exp(0,28/0,1) \gg \exp(-0,04/0,1) \gg \exp(-0,12/0,1)$$

Passo 2: normalizar para obter probabilidades. A ação Norte terá probabilidade dominante, mas Leste ainda terá probabilidade significativamente maior que Sul ou Oeste.

Interpretação: Mesmo durante a exploração, o agente tende a escolher ações promissoras com maior frequência. Diferentemente do ε -greedy, ações claramente ruins são evitadas naturalmente, sem necessidade de tratamento especial.

Isso resulta em uma exploração mais eficiente, especialmente em ambientes com muitas ações possíveis.

6.4 Estratégia por funções de exploração

Outra técnica para incentivar exploração é usar um bônus finito na **seleção de ações**. Para q-estados pouco visitados, usamos:

$$f(u, n) = u + \frac{k}{\max(1, n)}, \quad k > 0,$$

onde $u = Q(s, a)$ e $n = N(s, a)$ é o número de visitas ao par, inicialmente zero. Assim, o bônus de um par nunca visitado é k , não infinito. Para $n \geq 1$, essa função coincide com $u + k/n$. O agente seleciona:

$$a \in \arg \max_b f(Q(s, b), N(s, b)).$$

Após executar a , incrementa $N(s, a)$ e aplica $Q(s, a) \leftarrow Q(s, a) + \alpha[y - Q(s, a)]$, com o alvo **usual** do Q-learning: $y = r$ se s' for terminal e $y = r + \gamma \max_{a'} Q(s', a')$ caso contrário. O bônus não entra no alvo; Q continua estimando o retorno das recompensas originais.

Esse bônus representa otimismo sobre pares pouco visitados, mas a contagem é apenas uma aproximação da incerteza. Ele não garante, sozinho, exploração infinita nem menor arrependimento que ε -greedy. Se o bônus também fosse incluído no alvo, teríamos uma variante que altera os valores aprendidos durante o treinamento e exigiria análise própria; a definição $f(u, 0) = +\infty$ nesse alvo poderia produzir valores Q infinitos. Na versão apresentada aqui, não há propagação do bônus pelo alvo para estados predecessores.

6.4.1 Exemplo resolvido

Este exemplo ilustra a seleção de ação com função de exploração. Considere o mesmo agente em $s = (1, 1)$, agora usando $f(u, n) = u + k/n$ com $k = 1, 0$. Os Q-valores e contadores de visitas acumulados até o momento são:

	Norte	Sul	Leste	Oeste
$Q((1, 1), a)$	0,41	-0,12	0,28	-0,04
$N((1, 1), a)$	50	3	12	1

Passo 1: calcular $f(Q, N)$ para cada ação.

$$\begin{aligned} f(\text{Norte}) &= 0,41 + 1,0/50 = 0,41 + 0,02 = 0,430 \\ f(\text{Sul}) &= -0,12 + 1,0/3 \approx -0,12 + 0,33 = 0,213 \\ f(\text{Leste}) &= 0,28 + 1,0/12 \approx 0,28 + 0,08 = 0,363 \\ f(\text{Oeste}) &= -0,04 + 1,0/1 = -0,04 + 1,00 = 0,960 \end{aligned}$$

Passo 2: selecionar a ação que maximiza f .

$$a = \arg \max_a f(Q((1, 1), a), N((1, 1), a)) = \text{Oeste}$$

Interpretação: embora Oeste tenha o segundo pior Q-valor (-0,04), ela foi visitada apenas 1 vez e recebe um bônus de exploração alto (+1,0). O agente racionalmente a prefere a Norte (ação aparentemente melhor mas já muito visitada), pois a incerteza sobre Oeste ainda é grande. À medida que Oeste for visitada mais vezes, seu bônus diminuirá e Norte voltará a ser preferida, sem que o parâmetro ε precise ser ajustado manualmente.

Comparação direta com ε -greedy: com $\varepsilon = 0,1$, a probabilidade de escolher Oeste seria de apenas 0,025, independentemente de quantas vezes ela foi visitada. A função de exploração direciona o agente para pares pouco visitados no estado atual; a estratégia ε -greedy distribui a parte exploratória uniformemente entre as ações desse estado. Nenhuma delas implica visitas uniformes entre estados.

Pergunta 6

Compare as estratégias ε -greedy e funções de exploração nos seguintes aspectos: (a) como a intensidade da exploração diminui (ou não) ao longo do treinamento; (b) como o agente decide quais regiões do espaço de estados explorar; (c) em que condições cada estratégia pode produzir menor arrependimento (*regret*) total. Não há uma ordenação geral apenas pelas definições. Use exemplos concretos no Mundo Grade para ilustrar cada ponto.

7 Q-Learning com Aproximação Linear

7.1 Limitações do Q-learning tabular

O Q-learning tabular tem uma limitação prática fundamental: ele requer uma entrada na tabela para cada par (s, a) . Em problemas realistas, o número de estados pode ser astronomicamente grande.

A Figura 9 mostra um ambiente simplificado inspirado no Pac-Man. Um estado é definido por três componentes principais: a posição do agente, a posição de cada fantasma e o conjunto de pílulas ainda presentes no labirinto.

Mesmo nesse cenário reduzido, o número de estados já cresce rapidamente. Suponha que existam aproximadamente 20 células livres no labirinto. O agente pode ocupar qualquer uma delas, e cada um dos dois fantasmas também pode ocupar qualquer célula livre. Além disso, com 5 pílulas, cada uma pode estar presente ou ausente, resultando em $2^5 = 32$ configurações possíveis.

Ignorando restrições como colisões ou estados inválidos, o número total de estados é aproximadamente:

$$20 \cdot 20 \cdot 20 \cdot 32 = 256.000$$

Em um Pac-Man realista, com labirintos maiores, mais fantasmas e dezenas de pílulas, o espaço de estados explode combinatoriamente e pode facilmente superar 10^{27} . Manter uma tabela desse tamanho em memória é impossível; visitar cada entrada suficientemente durante o treinamento, igualmente impossível.

Além do problema de memória, há o problema da **generalização**: no Q-learning tabular, o agente aprende sobre cada estado-ação de forma completamente independente. Se ele descobre que um estado particular é ruim, essa informação não é automaticamente transferida para estados estruturalmente similares. O agente precisa visitar cada estado ruim individualmente para aprender a evitá-lo. Por exemplo, considere a Figura 10. Suponha que o Pac-Man aprende que o estado “fantasma a 1 célula de distância, parede à esquerda” é muito ruim. No Q-learning tabular, esse aprendizado nada diz sobre o estado “fantasma a 1 célula de distância, parede à direita”, mesmo que ambos sejam igualmente perigosos. São estados distintos na tabela, com entradas completamente independentes. A solução exige uma representação que capture as *similaridades estruturais* entre estados.

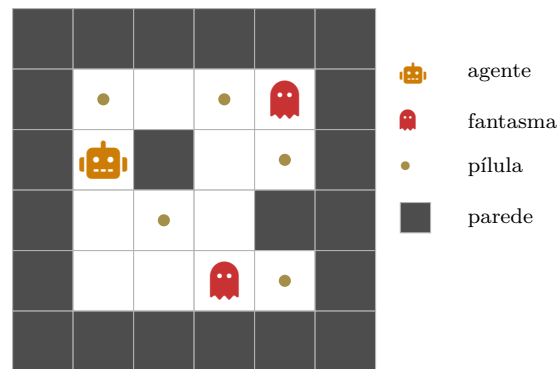


Figura 9: Ambiente simplificado inspirado no Pac-Man. Um estado é definido pela posição do agente (A), pela posição de cada fantasma (F) e pelo conjunto de pílulas restantes. Mesmo nesse cenário pequeno, o número de estados cresce rapidamente.

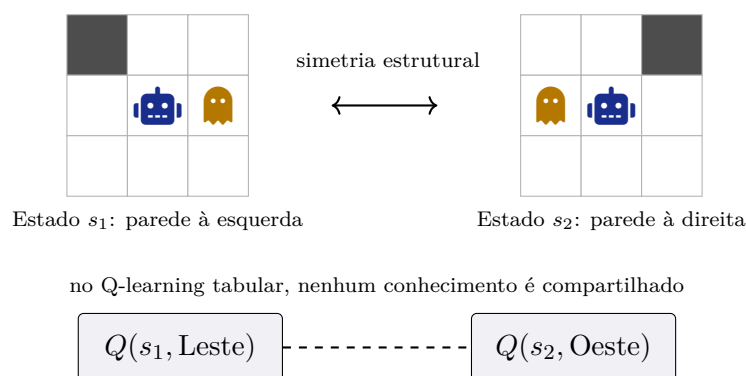


Figura 10: Dois estados estruturalmente semelhantes no Pac-Man. Em ambos, o agente está a uma célula do fantasma e encurralado por uma parede lateral. No Q-learning tabular, aprender que um deles é ruim não implica aprender nada sobre o outro.

Pergunta 7

Explique por que o Q-learning tabular não consegue generalizar o aprendizado entre estados similares. Por que isso é uma limitação prática severa em domínios como o Pac-Man, e não um problema no Mundo Grade (com apenas 11 estados)?

7.2 Características e representação

A solução é representar cada estado-ação não como uma entrada isolada em uma tabela, mas como um **vetor de características** (*features*). Uma característica é uma função $f_i(s, a)$ que mapeia um par estado-ação para um número real, capturando um aspecto relevante daquele par.

No contexto do Pac-Man, exemplos de características incluem:

- Distância para a pastilha mais próxima.
- Recíproco da distância para o fantasma mais próximo (valores maiores indicam fantasma próximo).
- Número de fantasmas a até 2 células de distância.

- Indicador binário: há uma cápsula de poder disponível?

O vetor de características para um par (s, a) é $\mathbf{f}(s, a) = [f_1(s, a), \dots, f_n(s, a)]$, com dimensão n , tipicamente muito menor do que o número de estados.

O processo de definir quais características usar é chamado de **engenharia de características** (*feature engineering*). Em aproximação linear, essa etapa é decisiva: o agente só consegue aprender regularidades que estejam, de algum modo, expressas nas features. Se uma propriedade importante do domínio não aparece na representação, o algoritmo não tem como “descobri-la” apenas ajustando pesos.

Boas características costumam satisfazer alguns critérios práticos:

- **Relevância:** a feature deve estar relacionada ao retorno futuro. No Pac-Man, a distância até a pastilha mais próxima e a proximidade de fantasmas são relevantes porque afetam diretamente recompensa e risco.
- **Capacidade de discriminação:** estados que exigem decisões diferentes não podem parecer iguais para o agente. Se dois estados possuem o mesmo vetor de features, a aproximação linear tenderá a atribuir valores semelhantes a eles, mesmo que um seja seguro e o outro leve à morte.
- **Generalização útil:** estados realmente parecidos devem compartilhar representação semelhante, para que a experiência em um deles ajude o aprendizado em outros. Esse é o principal ganho em relação ao método tabular.
- **Escala compatível:** features com magnitudes muito diferentes podem distorcer o aprendizado. Por isso, é comum normalizá-las para faixas semelhantes, como $[0, 1]$.

Há também um limite importante. Podemos sempre melhorar o desempenho adicionando features cada vez mais específicas, como “há um fantasma exatamente duas células à frente” ou “o agente está na mesma coluna do objetivo”. Isso pode funcionar, mas embute mais conhecimento humano na solução. Se exagerarmos, a engenharia de características deixa de ser uma representação compacta do problema e passa a ser uma codificação quase manual da política desejada.

Em resumo, a engenharia de características deve equilibrar dois objetivos opostos: *agrupar o que deve ser tratado de forma parecida e separar o que precisa ser distinguido*. Esse equilíbrio é uma das partes mais delicadas de todo o Q-learning com aproximação linear.

7.3 Engenharia de características: o verdadeiro gargalo

A introdução de características resolve dois problemas graves do Q-learning tabular: a explosão do espaço de estados e a ausência de generalização. No entanto, ela introduz um novo desafio, que passa a ser central: escolher uma boa representação. Em aproximação linear, o sucesso do agente depende menos da mecânica da atualização e mais de *quais aspectos do estado e da ação foram tornados visíveis* pelas features. Em outras palavras: ao abandonar a tabela, trocamos o problema de armazenar tudo pelo problema de decidir *o que merece ser representado*.

Essa mudança de perspectiva é profunda. No Q-learning tabular, dois pares estado-ação distintos sempre têm valores independentes: se o agente quiser tratá-los de forma parecida, isso terá de emergir apenas após muita experiência. Já na aproximação linear,

o compartilhamento de informação é imposto pela própria representação. Se dois pares (s, a) e (s', a') produzem vetores de características semelhantes, então seus valores Q tenderão a ser semelhantes também, pois ambos serão calculados a partir dos mesmos pesos. Isso é uma força e, ao mesmo tempo, um risco: a aproximação linear generaliza automaticamente, mas só generaliza *bem* se a representação capturar as distinções corretas do domínio.

Uma boa engenharia de características deve, portanto, satisfazer dois objetivos em tensão. Primeiro, deve promover generalização útil: estados estruturalmente semelhantes devem compartilhar informação, de modo que a experiência adquirida em um caso ajude em muitos outros. Segundo, deve preservar capacidade de discriminação: estados que exigem decisões diferentes não podem parecer iguais para o agente. Se a representação colapsa situações que deveriam ser distinguidas, o algoritmo será forçado a atribuir valores parecidos a ações que, no ambiente real, têm consequências muito diferentes.

Considere o caso de um agente em um Mundo Grade com um objetivo positivo, um estado terminal negativo e um obstáculo. Suponha que as features escolhidas codifiquem apenas “distância ao objetivo”. À primeira vista, isso parece razoável: aproximar-se do objetivo de fato costuma ser desejável. Mas essa representação pode ser insuficiente. Dois estados podem estar igualmente próximos do objetivo e, ainda assim, exigir ações opostas: em um deles, mover-se para a direita pode levar ao prêmio; em outro, pode levar diretamente ao estado terminal negativo. Se ambos recebem vetores de características muito parecidos, a aproximação linear terá dificuldade para atribuir valores diferentes a essas ações. O erro não estará na regra de atualização, mas na pobreza da representação.

Esse ponto revela por que a engenharia de características é o verdadeiro gargalo da abordagem. Quando as features são bem escolhidas, a aproximação linear pode aprender rapidamente, generalizar para estados nunca visitados e exigir muito menos experiência do que o método tabular. Quando são mal escolhidas, o agente aprende uma política sistematicamente distorcida, mesmo após muito treinamento. Assim, no Q-learning aproximado, a pergunta decisiva deixa de ser apenas “como atualizar os valores?” e passa a incluir “quais propriedades do problema precisam ser explicitamente codificadas para que a função aproximada consiga representar uma boa política?”

Há ainda um trade-off importante. Podemos sempre tentar corrigir uma representação ruim adicionando novas features mais específicas. Por exemplo, além da distância ao objetivo, podemos incluir indicadores de proximidade de estados perigosos, presença de obstáculos, alinhamento com o objetivo ou risco de colisão com fantasmas. Isso tende a melhorar o desempenho, mas tem um custo: quanto mais específicas e numerosas forem as features, mais conhecimento de domínio estamos embutindo manualmente no agente. Em casos extremos, a engenharia de características pode se aproximar de uma codificação parcial da própria solução. A aproximação linear continua sendo uma forma de aprendizado, mas sua qualidade depende fortemente da qualidade desse “vocabulário” fornecido ao agente.

Em resumo, a aproximação linear não elimina o problema de modelagem; ela o desloca. No lugar de armazenar explicitamente um valor para cada par estado-ação, passamos a construir uma linguagem de descrição do ambiente. Aprender bem depende, então, de duas coisas: ajustar corretamente os pesos e, antes disso, escolher características que revelem ao agente o que realmente importa no domínio.

7.4 De características de estado para características de estado-ação

Ao introduzir aproximação linear, é natural pensar primeiro em características do estado. Afinal, muitas propriedades relevantes parecem depender apenas da situação em que o agente se encontra: distância até a pastilha mais próxima, proximidade do fantasma mais próximo, número de cápsulas restantes, presença de obstáculo à frente, e assim por diante. No entanto, o objeto que queremos aproximar no Q-learning não é uma função apenas do estado, mas sim a função $Q(s, a)$, isto é, o valor de *executar uma ação específica* em um estado específico. Por isso, no fim das contas, a representação precisa distinguir não apenas *como é o estado*, mas também *qual ação está sendo avaliada*.

Essa distinção é importante porque a mesma propriedade do estado pode ter efeitos diferentes dependendo da ação considerada. No Pac-Man, por exemplo, “estar perto de uma pastilha” é uma informação relevante, mas seu efeito depende do movimento que está sendo avaliado: se a ação aproxima o agente da pastilha, isso tende a aumentar o valor; se a ação o afasta, o efeito pode ser o oposto. Da mesma forma, “fantasma próximo” não tem a mesma implicação para Norte, Sul, Leste e Oeste: uma dessas ações pode ser fuga, outra pode ser colisão iminente. Portanto, uma representação baseada apenas em features do estado, sem distinguir ações, seria incapaz de atribuir pesos diferentes a consequências diferentes.

A maneira mais simples de resolver esse problema é construir features de estado-ação a partir de um conjunto base de features de estado. Suponha que tenhamos n características de estado:

$$g_1(s), g_2(s), \dots, g_n(s)$$

e um conjunto de ações $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$. A partir delas, podemos formar um vetor de características de estado-ação com $n \times m$ componentes. A ideia é organizar esse vetor em blocos, um bloco para cada ação. Ao avaliar o par (s, a) , copiamos os valores das features do estado apenas para o bloco correspondente à ação a e zeramos os demais blocos.

Por exemplo, suponha que existam apenas duas features de estado:

- $g_1(s)$ = distância normalizada até a pastilha mais próxima
- $g_2(s)$ = proximidade do fantasma mais próximo

e quatro ações possíveis: Norte, Sul, Leste e Oeste. Então o vetor de características para o par (s, Norte) pode ser escrito como:

$$\mathbf{f}(s, \text{Norte}) = [g_1(s), g_2(s), 0, 0, 0, 0, 0, 0]$$

enquanto o vetor para (s, Leste) seria:

$$\mathbf{f}(s, \text{Leste}) = [0, 0, 0, 0, g_1(s), g_2(s), 0, 0]$$

Com essa construção, a aproximação linear passa a ter, na prática, um conjunto distinto de pesos para cada ação. Se escrevermos

$$Q(s, a) = \mathbf{w} \cdot \mathbf{f}(s, a),$$

então os pesos associados ao bloco da ação Norte aprendem o quanto cada feature importa ao avaliar Norte; os pesos do bloco da ação Leste aprendem o quanto essas mesmas

features importam ao avaliar Leste; e assim por diante. Isso permite que uma mesma característica, por exemplo, “fantasma próximo”, contribua negativamente para uma ação e positivamente para outra, dependendo do que essa ação tende a causar.

Essa transformação permite começar com uma descrição intuitiva do estado e, a partir dela, obter uma representação compatível com a função $Q(s, a)$. Conceitualmente, isso equivale a aprender um pequeno modelo linear separado para cada ação, todos compartilhando o mesmo vocabulário de features do estado. Assim, o projetista pode pensar primeiro em *o que importa no estado* e depois deixar que o formalismo transforme isso em *como cada ação deve ser avaliada nesse estado*. Essa distinção também ajuda a esclarecer por que o Q-learning com aproximação linear ainda pode diferenciar ações mesmo quando parte de um mesmo estado. O que muda de uma ação para outra não é apenas o valor numérico final de Q , mas o próprio subconjunto de pesos que entra em jogo na combinação linear. Em resumo: as features de estado descrevem a situação; as features de estado-ação permitem avaliar o impacto potencial de cada ação nessa situação.

7.5 Por que normalizar as características?

Ao trabalhar com aproximação linear, não basta escolher *quais* características usar; também é importante cuidar da *escala numérica* em que elas são representadas. Em princípio, qualquer grandeza relevante do domínio pode ser usada como feature: distância até o objetivo, número de fantasmas próximos, quantidade de pílulas restantes, indicador de obstáculo, entre outras. O problema é que essas grandezas podem viver em ordens de magnitude muito diferentes. Por exemplo, uma característica binária pode assumir apenas os valores 0 ou 1, enquanto uma distância pode variar de 0 a 50. Se colocarmos ambas diretamente no vetor de features, a característica de maior escala tende a dominar a combinação linear e também a magnitude das atualizações dos pesos.

Esse efeito aparece diretamente na regra de atualização:

$$w_i \leftarrow w_i + \alpha \cdot \delta \cdot f_i(s, a)$$

Observe que a própria *feature* $f_i(s, a)$ multiplica o erro temporal δ . Isso significa que, se uma característica assume valores sistematicamente muito maiores do que as demais, seu peso tenderá a sofrer atualizações maiores, independentemente de sua real importância conceitual no problema. Em consequência, o processo de aprendizado pode ficar desequilibrado: parte das diferenças entre pesos passará a refletir apenas diferenças de escala numérica entre as *features*, e não diferenças reais de relevância.

Por esse motivo, é prática comum *normalizar* as características, isto é, transformá-las para faixas comparáveis, tipicamente entre 0 e 1 ou entre -1 e 1 . No GridWorld, por exemplo, em vez de usar diretamente a coordenada x do agente ou a distância de Manhattan até o objetivo, podemos dividi-las por seus valores máximos possíveis na grade. Isso preserva a informação sem deixar que uma *feature* domine as outras apenas por ser numericamente maior.

Normalizar uma característica não altera *o que ela mede*; altera apenas a escala em que essa informação é apresentada ao agente. O objetivo é fazer com que os pesos reflitam mais fielmente a importância relativa das features, e não diferenças arbitrárias de unidade de medida. Em problemas com aproximação linear, isso costuma tornar o aprendizado mais estável, mais interpretável e menos sensível à escolha da taxa de aprendizado α . Considere, por exemplo, duas features: $f_1(s, a)$ igual à distância até a pastilha mais próxima, variando entre 0 e 20, e $f_2(s, a)$ igual a um indicador binário de perigo imediato,

variando entre 0 e 1. Mesmo que ambas sejam igualmente relevantes para o retorno futuro, a primeira tenderia a exercer influência numérica muito maior caso fosse usada sem normalização. Ao reescalá-la, por exemplo dividindo por 20, tornamos mais justa a competição entre as duas no processo de ajuste dos pesos.

Em resumo, a normalização é um detalhe técnico com forte impacto prático. Ela não resolve o problema de escolher boas features, mas evita que features razoáveis se tornem difíceis de usar apenas porque foram expressas em escalas incompatíveis.

7.6 A Q-função aproximada

No Q-learning com aproximação linear, a Q-função não é mais uma tabela, mas uma combinação linear das características:

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a) = \mathbf{w} \cdot \mathbf{f}(s, a)$$

onde $\mathbf{w} = [w_1, w_2, \dots, w_n]$ é um vetor de pesos a serem aprendidos. O objetivo do aprendizado muda: em vez de aprender o valor de cada entrada da tabela, o agente aprende um único vetor de pesos de dimensão n , compartilhado entre todos os estados. Assim, experiências em um estado influenciam automaticamente as estimativas de Q para estados similares; o ciclo de generalização se fecha aqui.

A Figura 11 contrasta as duas representações, destacando como a função de valor de ação $Q(s, a)$ é armazenada e atualizada em cada caso. No Q-learning tabular, cada par estado-ação possui uma entrada explícita em uma tabela, o que permite atualizações locais e independentes: ao observar uma transição (s, a, r, s') , apenas o valor correspondente a (s, a) é modificado. Essa abordagem é simples e direta, mas não escala bem, pois o número de entradas cresce rapidamente com o tamanho do espaço de estados e ações. Já na aproximação linear, $Q(s, a; \theta)$ é representado como uma função parametrizada, tipicamente uma combinação linear de características (*features*) do par (s, a) . Nesse caso, a atualização não afeta apenas um único par estado-ação, mas todos aqueles que compartilham características semelhantes, promovendo generalização. Como consequência, o agente pode aprender sobre estados nunca visitados diretamente, mas a atualização deixa de ser local e passa a envolver ajustes nos parâmetros θ . Assim, a figura evidencia um trade-off fundamental: a representação tabular oferece simplicidade e precisão local, enquanto a aproximação linear permite escalabilidade e generalização, ao custo de introduzir dependências entre diferentes estados e ações.

Essa representação tem duas consequências importantes:

- **Generalização automática:** dois pares estado-ação com características similares terão valores Q similares, porque Q é determinado pelas mesmas características com os mesmos pesos. O agente generaliza automaticamente para estados não visitados.
- **Eficiência:** em vez de aprender $|S| \times |A|$ valores independentes, o agente aprende apenas n pesos. Mesmo com poucos episódios de treinamento, o aprendizado de um peso sobre uma característica melhora as estimativas de Q para todos os estados que possuem essa característica.

Uma aproximação linear tende a ser adequada quando as características capturam as dimensões relevantes do valor de estado-ação e quando esse valor varia de forma aproximadamente proporcional a elas. Por exemplo, no Pac-Man, a distância ao fantasma e a distância à pastilha podem capturar fatores importantes de risco e recompensa.

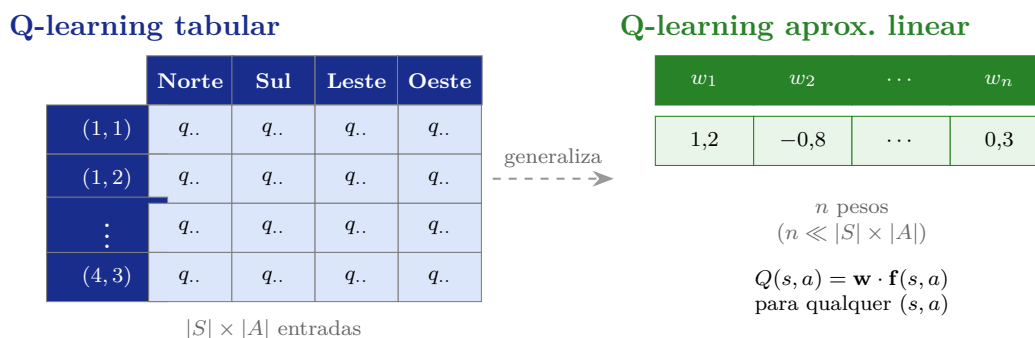


Figura 11: Comparação entre as representações tabular e linear. À esquerda, o Q-learning tabular armazena um valor independente para cada par (s, a) . À direita, o Q-learning com aproximação linear armazena apenas n pesos compartilhados, permitindo generalizar para estados não visitados que compartilham características com estados já vistos.

- As características capturam as dimensões relevantes do valor de estado-ação.
- O valor de Q varia de forma aproximadamente proporcional às características, como quando maior proximidade de um fantasma representa maior perigo.

Sob essas condições, a aproximação linear pode ser vista como uma forma de generalização supervisionada dentro do AR: a cada transição, o agente está ajustando um hiperplano (em espaço n -dimensional) para melhor aproximar os valores verdadeiros. As características que o ser humano fornece funcionam como um vocabulário que o agente usa para descrever o mundo. Na aproximação linear, o agente aprende os pesos $\mathbf{w}$, compartilhados por todos os pares estado-ação. A estimativa de cada $Q(s, a)$ varia conforme o vetor de características $\mathbf{f}(s, a)$; portanto, a representação determina tanto o que pode ser generalizado quanto o que pode ser distinguido.

7.7 Exemplo: projetando e refinando características no Mundo Grade

Até aqui, discutimos a aproximação linear em termos gerais. Mas, na prática, como se escolhem features para um problema concreto? Nesta subseção, usamos o Mundo Grade para mostrar um ponto crucial: *features aparentemente razoáveis podem ainda ser insuficientes para produzir uma boa política*. O objetivo não é apenas exibir uma lista de características possíveis, mas mostrar o processo iterativo real de engenharia de features: propor uma representação, observar suas limitações e refiná-la.

Considere o Mundo Grade com um objetivo positivo no canto superior direito, um estado terminal negativo logo abaixo dele e uma célula bloqueada. Uma primeira ideia natural é usar apenas características relacionadas à proximidade do objetivo. Por exemplo:

- distância horizontal até o objetivo;
- distância vertical até o objetivo;
- distância de Manhattan até o objetivo.

À primeira vista, essas features parecem sensatas. Afinal, em muitos problemas de navegação, aproximar-se do objetivo é desejável. Além disso, elas têm duas vantagens

claras: são fáceis de calcular e promovem generalização entre estados que estão a distâncias semelhantes do alvo. Se dois estados estão aproximadamente à mesma distância do objetivo, é plausível que tenham valores parecidos.

No entanto, essa representação deixa de fora informações importantes do ambiente. Em particular, ela não distingue adequadamente situações em que *aproximar-se do objetivo é perigoso*. No Mundo Grade, mover-se para a direita pode ser excelente em alguns estados, pois aproxima o agente do terminal positivo, mas pode ser péssimo em outros, pois o expõe ao terminal negativo ou a regiões em que pequenos desvios estocásticos se tornam arriscados. Se a representação codifica apenas “proximidade do objetivo”, então estados que deveriam ser tratados de forma diferente acabam parecendo excessivamente semelhantes para o agente.

Isso produz um tipo específico de erro: a aproximação linear pode aprender corretamente que a ação Leste é boa em vários estados próximos do objetivo e, por compartilhamento de pesos, passar a superestimar essa mesma ação em estados onde ela deveria ser evitada. O problema não é que o algoritmo esteja atualizando os pesos incorretamente. O problema é que a representação não oferece graus de liberdade suficientes para separar situações “próximas do objetivo e seguras” de situações “próximas do objetivo, mas perigosas”.

Uma forma de melhorar essa representação é adicionar features que codifiquem alguma estrutura mais específica do ambiente. Por exemplo, além das três distâncias acima, poderíamos incluir:

- um indicador binário de que o agente está na mesma coluna do objetivo;
- um indicador binário de que o agente está na mesma linha do objetivo.

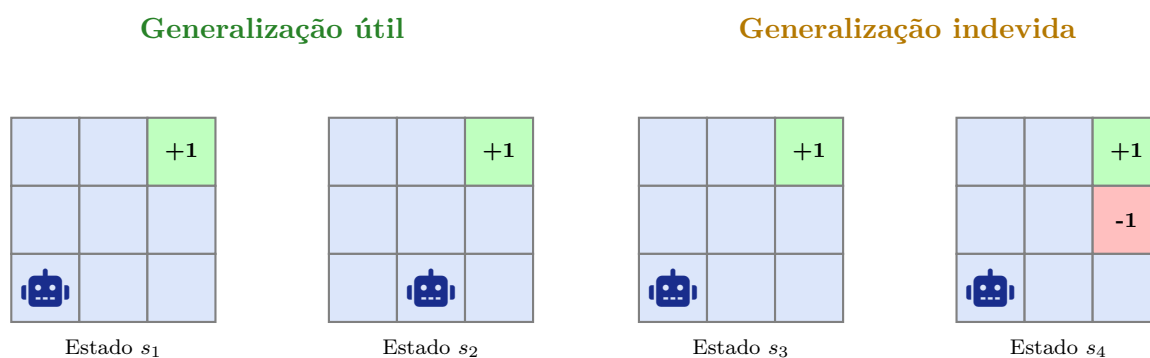
Essas novas features permitem capturar regularidades que a simples distância não expressa. Estar alinhado horizontal ou verticalmente com o objetivo pode alterar significativamente o valor das ações disponíveis, sobretudo quando existem estados terminais negativos ou obstáculos próximos. Com isso, a função aproximada passa a ter mais capacidade para distinguir contextos em que “ir em direção ao objetivo” é seguro daqueles em que essa mesma tendência precisa ser moderada.

A lição central não é que essas features específicas sejam sempre as melhores, mas sim que a qualidade da aproximação depende da capacidade de a representação separar situações que exigem decisões diferentes. Distâncias ao objetivo capturam uma parte importante do problema, mas não toda. Quando a política aprendida apresenta erros sistemáticos, uma pergunta útil é: *que aspecto relevante do ambiente não está sendo visto pelo vetor de features?* Mesmo assim, o refinamento não resolve tudo. Ainda pode haver erros residuais, porque a aproximação linear continua impondo uma forma funcional limitada: ela combina as features de maneira aditiva e linear. Em problemas onde o valor depende de interações mais complexas (por exemplo, “ir para a direita é bom apenas se eu não estiver na linha do estado terminal negativo”), a classe linear pode continuar insuficiente, ainda que as features sejam melhores do que antes.

Esse exemplo ilustra o ciclo típico da engenharia de características em aprendizado por reforço: começamos com uma representação compacta e intuitiva, testamos o comportamento resultante, diagnosticamos onde a política erra e então refinamos as features para tornar visíveis distinções relevantes do domínio. A aproximação linear funciona bem quando esse vocabulário é expressivo o suficiente para capturar as regularidades importantes, mas simples o bastante para ainda generalizar e escalar bem.

A Figura 10 e a Figura 12 revelam as duas faces do problema. No método tabular, não há compartilhamento algum de conhecimento, mesmo entre estados muito parecidos. Na aproximação linear, o compartilhamento passa a existir, o que é uma grande vantagem, mas ele depende inteiramente da qualidade da representação escolhida. O objetivo da engenharia de características é, precisamente, fazer com que esse compartilhamento ocorra nos pares de estados em que ele é legítimo e deixe de ocorrer quando pequenas diferenças locais alteram drasticamente a qualidade de uma ação.

A Figura 12 ilustra um ponto central da aproximação linear: *nem toda generalização é desejável*. No painel da esquerda, temos um caso de generalização útil: dois estados estruturalmente semelhantes, nos quais o agente está em situação análoga em relação ao objetivo, sem risco local relevante. Nessa situação, faz sentido que a experiência adquirida em um estado influencie a estimativa de valor no outro. Já no painel da direita, a similaridade é enganosa: embora os dois estados pareçam igualmente promissores sob features simplificadas baseadas apenas em distância ao objetivo, um deles contém um risco local importante: o estado terminal negativo logo à frente. Se a representação não consegue expressar essa diferença, a função aproximada tenderá a transferir conhecimento de forma indevida, atribuindo valores parecidos a situações que deveriam ser tratadas de forma diferente. A figura resume, portanto, o critério decisivo da engenharia de características: *generalizar onde há estrutura compartilhada real e separar onde pequenas diferenças mudam a decisão ótima*.



Mensagem central: a aproximação linear compartilha conhecimento entre estados, mas isso só ajuda quando a representação agrupa situações que realmente deveriam ter valores semelhantes.

Figura 12: Boas e más generalizações na aproximação linear. À esquerda, os estados s_1 e s_2 são estruturalmente semelhantes: ambos colocam o agente em uma situação análoga em relação ao objetivo positivo, de modo que compartilhar representação entre eles é desejável. À direita, os estados s_3 e s_4 parecem parecidos se olharmos apenas para a proximidade do objetivo, mas diferem em um aspecto crucial: em s_4 , há um estado terminal negativo logo abaixo do objetivo. Se as features não capturam essa diferença local, a função aproximada pode transferir conhecimento de forma indevida.

Pergunta 8

No Mundo Grade, um agente com aproximação linear usa apenas features de proximidade ao objetivo e aprende uma política que, em alguns estados, insiste em mover-se para a direita mesmo quando isso o aproxima perigosamente do estado terminal negativo.

Explique por que esse erro pode ocorrer mesmo que a regra de atualização dos pesos esteja correta. Em sua resposta, discuta: **(a)** o que a representação atual consegue capturar; **(b)** o que ela não consegue distinguir; **(c)** como isso produz uma generalização indevida; e **(d)** que tipo de feature adicional poderia tornar visível ao agente a diferença entre “aproximar-se do objetivo com segurança” e “aproximar-se do objetivo com risco”.

7.8 Regra de atualização dos pesos

A cada transição (s, a, r, s') , o Q-learning aproximado atualiza os pesos. Na fórmula abaixo, o alvo é $r + \gamma \max_{a'} Q(s', a')$ para sucessores não terminais; se s' for terminal, usamos apenas r , sem avaliar ações nesse estado. A regra de atualização é:

$$\delta = [r + \gamma \cdot \max_{a'} Q(s', a')] - Q(s, a) \quad (\text{erro de diferença temporal})$$

$$w_i \leftarrow w_i + \alpha \cdot \delta \cdot f_i(s, a) \quad \text{para cada } i = 1, \dots, n$$

A interpretação depende do sinal da feature:

- A variação de cada peso é $\Delta w_i = \alpha \delta f_i(s, a)$. Para $\delta > 0$, um peso aumenta se $f_i > 0$ e diminui se $f_i < 0$; para $\delta < 0$, os sentidos se invertem.
- Se $f_i(s, a) = 0$, o peso não é alterado por esta transição.
- Mantendo o vetor de features fixo, a previsão do par atual varia por

$$\Delta Q(s, a) = \alpha \delta \sum_i f_i(s, a)^2.$$

Portanto, ela muda na direção indicada pelo erro, mesmo com features negativas. Uma taxa excessiva pode causar sobrecorreção.

Esse mecanismo distribui a correção entre características presentes na previsão; não identifica quais delas causaram o resultado. Como os pesos são compartilhados, a atualização também interfere nas previsões de outros pares estado-ação.

7.8.1 Exemplo resolvido

Este exemplo ilustra a atualização de pesos no Pac-Man. Considere $Q(s, a) = w_{\text{DOT}} \cdot f_{\text{DOT}}(s, a) + w_{\text{GST}} \cdot f_{\text{GST}}(s, a)$, onde as duas features são $1/(1 + d)$ para a pastilha e o fantasma mais próximos. Medimos as distâncias da posição projetada após a ação, mantendo os demais elementos como observados em s ; se o elemento não existir, a feature vale zero. Assim, as features distinguem ações e são finitas para $d = 0$. Para a ação Norte, considere os seguintes pesos e valores de features: $w_{\text{DOT}} = 4,0$, $w_{\text{GST}} = -1,0$; $f_{\text{DOT}} = 0,5$, $f_{\text{GST}} = 1,0$.

Passo 1: calcular $Q(s, \text{Norte})$ com os pesos atuais.

$$Q(s, \text{Norte}) = 4,0 \times 0,5 + (-1,0) \times 1,0 = 2,0 - 1,0 = 1,0$$

O agente executa Norte e morre: $r = -500$; s' é terminal e não há valor de continuação (alvo $y = r$).

Passo 2: calcular o erro temporal δ (com $\gamma = 0,9$).

$$\delta = [-500 + 0,9 \times 0] - 1,0 = -501,0$$

Passo 3: atualizar os pesos (com $\alpha = 0,01$).

$$w_{\text{DOT}} \leftarrow 4,0 + 0,01 \times (-501,0) \times 0,5 = 4,0 - 2,505 = 1,495$$

$$w_{\text{GST}} \leftarrow -1,0 + 0,01 \times (-501,0) \times 1,0 = -1,0 - 5,01 = -6,01$$

Interpretação: ambos os pesos foram alterados negativamente porque $\delta < 0$. O peso de f_{GST} foi mais fortemente penalizado porque $f_{\text{GST}} = 1,0 > f_{\text{DOT}} = 0,5$: a feature “fantasma próximo” tinha maior magnitude e recebeu uma correção maior. O peso da proximidade à pastilha também caiu, embora isso não demonstre que a pastilha tenha causado a morte. O exemplo ilustra interferência entre características, não identificação causal.

7.9 Relação com regressão linear e erro quadrático

Há uma conexão importante entre o Q-learning com aproximação linear e a regressão linear que você já conhece. Em regressão linear, ajustamos coeficientes para que a previsão do modelo fique o mais próxima possível de um valor-alvo observado. Aqui ocorre algo análogo: a função aproximada

$$Q(s, a) = \mathbf{w} \cdot \mathbf{f}(s, a)$$

faz o papel da previsão, e o valor

$$y = r + \gamma \max_{a'} Q(s', a')$$

faz o papel do alvo associado à transição observada. Em outras palavras, após observar a experiência (s, a, r, s') , queremos que a estimativa atual $Q(s, a)$ fique mais próxima do alvo de Bellman y .

Durante cada atualização, tratamos o alvo de Bellman como fixo e ajustamos apenas a previsão do par atual. Se $\mathbf{w}_t$ são os pesos antes do passo, definimos $y_t = r + \gamma \max_{a'} Q_{\mathbf{w}_t}(s', a')$ (ou $y_t = r$ no terminal) e o erro quadrático local:

$$L_t(\mathbf{w}) = \frac{1}{2} \left[y_t - Q_{\mathbf{w}}(s, a) \right]^2$$

O fator $\frac{1}{2}$ aparece apenas por conveniência algébrica; ele não altera o ponto de mínimo da função. Minimizar esse erro com y_t fixo significa ajustar os pesos para reduzir a discrepância entre a previsão atual e o alvo construído a partir da transição observada.

Se definirmos o erro temporal por

$$\delta = y - Q(s, a) = \left[r + \gamma \max_{a'} Q(s', a') \right] - Q(s, a),$$

então a atualização

$$w_i \leftarrow w_i + \alpha \cdot \delta \cdot f_i(s, a)$$

pode ser interpretada de maneira simples: cada peso é corrigido na direção que tende a reduzir o erro da previsão. Além disso, a correção é proporcional ao valor da feature correspondente. Isso faz sentido: se uma característica está muito ativa no par (s, a) , ela teve maior participação na previsão atual e, portanto, deve receber uma correção maior; se a característica vale zero, ela não contribuiu para aquela previsão e seu peso não precisa ser alterado.

Essa conexão ajuda a enxergar o Q-learning aproximado como uma forma de ajuste sucessivo de um modelo linear. A cada transição, o agente recebe um novo “exemplo de treinamento”: o vetor de características $\mathbf{f}(s, a)$ entra como entrada do modelo, e o alvo de Bellman entra como valor que a previsão deveria atingir. O aprendizado consiste em repetir esse processo muitas vezes, refinando gradualmente os pesos para que a função $Q(s, a)$ produza estimativas cada vez mais adequadas.

Formalmente, a regra é um passo de **semigradiente**: derivamos apenas a previsão $Q_{\mathbf{w}}(s, a)$ e não a dependência do alvo nos pesos. Ela é o gradiente negativo do erro local com y_t congelado, mas não o gradiente completo de $\frac{1}{2}[r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a') - Q_{\mathbf{w}}(s, a)]^2$. Mas, para a intuição desta disciplina, o ponto principal é outro: **o agente está ajustando uma função linear para aproximar, em cada transição, o alvo produzido pela equação de Bellman**. Nesse sentido, o Q-learning com aproximação linear combina duas ideias já conhecidas: previsão linear e atualização por erro.

Na regressão linear usual, o valor-alvo vem diretamente dos dados observados. No Q-learning aproximado, o alvo

$$r + \gamma \max_{a'} Q(s', a')$$

não é um valor externo fixo: ele depende da própria função Q atual. Por isso, o aprendizado em AR é mais instável e mais sutil do que em regressão supervisionada comum. Ainda assim, a analogia com regressão linear continua útil para entender a lógica da atualização dos pesos.

Pergunta 9

Suponha $Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a)$ com $w_1 = 2,0$ e $w_2 = -0,5$. Para a transição $(s, a, r = +0,5, s')$, os valores das características são $f_1(s, a) = 1,0$ e $f_2(s, a) = 0,0$, e $\max_{a'} Q(s', a') = 1,2$. Use $\alpha = 0,1$ e $\gamma = 0,9$. Calcule: **(a)** $Q(s, a)$ com os pesos atuais; **(b)** o alvo; **(c)** o erro temporal δ ; **(d)** os pesos atualizados w_1 e w_2 . Interprete por que w_2 não mudou.

7.10 O ônus e o bônus da aproximação linear

A aproximação linear traz um bônus inegável: torna viável aprender em domínios onde o Q-learning tabular simplesmente não escala. Em vez de armazenar um valor separado para cada par (s, a) , o agente aprende um número muito menor de parâmetros e ganha a capacidade de *generalizar*. Isso significa que a experiência adquirida em um estado pode influenciar a estimativa de valor em muitos outros estados semelhantes, inclusive estados

nunca visitados diretamente. Em domínios grandes, esse ganho é decisivo. Mas esse bônus vem acompanhado de um ônus: o sucesso do agente passa a depender fortemente da qualidade das features escolhidas.

No método tabular, a representação é exata: cada par estado-ação possui seu próprio valor, sem interferência direta de outros pares. Na aproximação linear, a representação deixa de ser exata e passa a depender de uma hipótese estrutural: a de que o valor $Q(s, a)$ pode ser bem aproximado por uma combinação linear das features escolhidas. Quando essa hipótese é razoável, o método funciona bem. Quando não é, o agente pode aprender políticas sistematicamente imperfeitas, mesmo após muito treinamento.

Além do problema de representação, há um problema de estabilidade: o Q-learning com aproximação linear **pode divergir**. A combinação de aproximação de funções, bootstrap e aprendizado off-policy não herda a garantia tabular; melhorar as features, por si só, não resolve essa questão (ver os capítulos 10–11 de Sutton and Barto, 2018).

Em termos de representação, a aproximação linear troca um problema de *memória e cobertura* por um problema de *representação*. No tabular, a dificuldade está em visitar tudo. Na aproximação linear, a dificuldade está em escolher um vocabulário de features que permita ao agente distinguir situações que realmente exigem decisões diferentes, sem perder a capacidade de generalizar entre situações semelhantes. O sucesso do método depende fortemente desse equilíbrio.

Há, portanto, um compromisso inevitável. Se usamos poucas features e muito genéricas, ganhamos simplicidade e forte generalização, mas corremos o risco de colapsar estados que deveriam ser distinguidos. Se adicionamos muitas features específicas, aumentamos a capacidade de discriminação e podemos corrigir erros da política, mas perdemos simplicidade, exigimos mais conhecimento de domínio e nos aproximamos de uma codificação manual da solução. O desafio prático da aproximação linear está justamente em encontrar um ponto intermediário entre essas duas extremidades.

O Q-learning tabular mantém valores separados e converge para Q^* sob as condições estabelecidas, mas não escala e não generaliza. O Q-learning com aproximação linear aprende *uma regra geral para muitos estados ao mesmo tempo*, o que escala melhor, mas só funciona tão bem quanto a representação escolhida permitir. Assim, a principal pergunta deixa de ser apenas “como atualizar Q ?” e passa a ser também “que estrutura do domínio precisa estar explícita nas features para que uma boa política possa ser representada?”. Essa é a razão pela qual a aproximação linear, embora poderosa, não deve ser vista como uma solução automática para o problema da explosão do espaço de estados. Ela é uma troca: abrimos mão da exatidão do método tabular para ganhar escalabilidade e generalização. Em muitos domínios, essa troca vale a pena. Mas ela só produz bons resultados quando as features escolhidas conseguem capturar, de forma suficientemente fiel, os fatores que realmente determinam a qualidade das ações.

8 Reward Shaping e Recompensas Intermediárias

8.1 Por que pensar em *reward shaping*?

Os algoritmos de aprendizado por reforço estudados até aqui (como Q-learning e SARSA) aprendem a partir das recompensas observadas durante a interação com o ambiente. Isso funciona bem quando o agente recebe sinais frequentes sobre a qualidade de seu comportamento. No entanto, em muitos problemas interessantes, as recompensas são *esparsas*: durante longos trechos da experiência, quase todas as transições produzem

recompensa zero ou um sinal muito pouco informativo. Nesses casos, o agente pode passar muitos episódios explorando quase às cegas antes de descobrir uma trajetória que leve a uma recompensa relevante.

Esse problema é particularmente grave no início do treinamento. Como o agente ainda sabe muito pouco, sua política de exploração o leva a executar ações quase aleatórias. Se o objetivo estiver distante ou for difícil de alcançar, o agente demorará a receber qualquer evidência de que certas direções são promissoras. Em consequência, o aprendizado pode ser muito lento: não porque a regra de atualização esteja errada, mas porque o sinal disponível para atualizá-la chega tarde demais e com pouca frequência.

Uma forma de atacar esse problema é incorporar, de maneira controlada, algum *conhecimento de domínio* ao processo de aprendizagem. Em muitos ambientes, mesmo sem conhecer a política ótima, um ser humano consegue formular intuições razoáveis sobre quais comportamentos parecem aproximar o agente da solução. No Mundo Grade, por exemplo, aproximar-se do estado terminal positivo e afastar-se do terminal negativo parece, em geral, desejável. Em jogos de navegação, mover-se em direção ao objetivo tende a ser melhor do que afastar-se dele. A ideia do *reward shaping* é explorar exatamente esse tipo de intuição.

De forma informal, *reward shaping* consiste em adicionar pequenos sinais intermediários de recompensa que *incentivam* transições consideradas promissoras e *desencorajam* transições consideradas pouco úteis. O objetivo não é mudar o problema fundamental que o agente deve resolver, mas sim tornar o sinal de aprendizado mais informativo durante o treinamento, especialmente nas fases iniciais. Em vez de esperar apenas pela recompensa “verdadeira” do ambiente, o agente passa a receber também um empurrão heurístico que pode acelerar a convergência.

Reward shaping é uma forma de dizer ao agente: “ainda não cheguei ao objetivo, mas esta transição parece um passo na direção certa”. Em ambientes de recompensa esparsa, esse tipo de sinal adicional pode reduzir bastante o tempo necessário para que o aprendizado comece a se organizar. A ideia é poderosa, mas exige cuidado. Se o sinal adicional for mal projetado, ele pode induzir o agente a preferir comportamentos que parecem bons localmente, mas que não ajudam de fato a resolver o problema. Por isso, a questão central não é apenas *como* adicionar recompensas intermediárias, mas *como fazê-lo sem distorcer a política ótima que desejamos aprender*. A próxima subseção apresenta essa ideia de forma mais precisa.

8.2 A ideia básica

A ideia mais simples de *reward shaping* é modificar o sinal usado na atualização do agente, acrescentando uma recompensa intermediária adicional às transições observadas. Em vez de atualizar os valores apenas com base na recompensa original do ambiente, o agente passa a usar uma *recompensa ajustada*:

$$r'(s, a, s') = r(s, a, s') + F(s, s')$$

onde $r(s, a, s')$ é a recompensa original e $F(s, s')$ é um termo adicional escolhido pelo projetista. Esse termo expressa uma heurística sobre a qualidade da transição: se passar de s para s' parece um passo na direção certa, $F(s, s')$ pode ser positivo; se parece um passo na direção errada, $F(s, s')$ pode ser negativo.

Com essa modificação, a atualização do Q-learning passa a usar:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r(s, a, s') + F(s, s') + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

A única diferença em relação à regra original é a presença do termo $F(s, s')$. Conceitualmente, ele funciona como um *empurrão heurístico*: além da recompensa “verdadeira” do ambiente, o agente recebe um pequeno sinal adicional que torna algumas transições mais atrativas e outras menos.

A motivação é clara. Em problemas de recompensa esparsa, a maior parte das transições produz pouco ou nenhum sinal útil. Assim, o agente pode levar muito tempo para perceber que certas trajetórias são promissoras. O *reward shaping* tenta reduzir esse atraso, fornecendo indícios mais frequentes sobre a direção desejável do comportamento. No Mundo Grade, por exemplo, podemos querer recompensar levemente transições que aproximam o agente do objetivo positivo e penalizar transições que o afastam ou o aproximam de regiões perigosas.

Sem *reward shaping*, o agente aprende apenas a partir das recompensas que o ambiente realmente fornece. Com *reward shaping*, o agente continua aprendendo a partir dessas recompensas, mas recebe também uma orientação adicional do modelador. A intenção é acelerar o aprendizado inicial, não substituir o problema por outro. Essa ideia, embora intuitiva, traz um risco importante. Se o termo adicional $F(s, s')$ for escolhido de forma ingênua, ele pode alterar não apenas a velocidade do aprendizado, mas também o comportamento final aprendido pelo agente. Em outras palavras, um *reward shaping* mal projetado pode fazer o agente otimizar um problema diferente daquele que realmente queríamos resolver.

Adicionar recompensas intermediárias arbitrariamente pode parecer uma boa ideia, mas pode distorcer a política aprendida. Por isso, em aprendizado por reforço, a pergunta correta não é apenas “que comportamentos quero incentivar?”, mas também “como incentivar esses comportamentos sem mudar a solução ótima do problema?”. Essa observação leva à forma mais importante e mais segura de *reward shaping*: o *reward shaping* baseado em potencial, apresentado a seguir.

8.3 *Reward shaping* baseado em potencial

A ideia básica do *reward shaping* é adicionar um termo heurístico à recompensa observada em cada transição. No entanto, se esse termo for escolhido de maneira arbitrária, ele pode alterar o problema que o agente está realmente resolvendo. Por isso, uma forma particularmente importante de *reward shaping* é a versão *baseada em potencial*, que introduz conhecimento de domínio de modo mais controlado.

Nessa abordagem, o termo adicional $F(s, s')$ não é definido livremente. Em vez disso, ele é construído a partir de uma função escalar $\Phi(s)$, chamada *função potencial*, que associa a cada estado uma medida heurística de quão promissor ele parece ser:

$$F(s, s') = \gamma\Phi(s') - \Phi(s)$$

onde γ é o fator de desconto do problema. Assim, a recompensa ajustada passa a ser:

$$r'(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s)$$

O sinal depende do desconto: $F(s, s') > 0$ se e somente se $\gamma\Phi(s') > \Phi(s)$. Aumentar o potencial não basta para obter bônus positivo. Por exemplo, com $\gamma = 0,9$, passar de 0,80 para 0,85 produz $F = 0,9(0,85) - 0,80 = -0,035$. Em outras palavras, a função potencial permite transformar uma intuição qualitativa (por exemplo, “estados mais próximos do objetivo parecem melhores”) em um sinal quantitativo que influencia o aprendizado passo a passo.

A função $\Phi(s)$ não precisa ser o valor verdadeiro do estado, nem uma estimativa perfeita de $V^*(s)$. Ela funciona apenas como uma heurística: um modo de dizer quais estados *parecem* mais promissores segundo o conhecimento de domínio disponível. Quanto melhor essa heurística refletir a estrutura real do problema, maior a chance de o *reward shaping* acelerar o aprendizado. A principal razão para dar destaque a essa forma particular de *reward shaping* é sua garantia de invariância da política ótima³. Quando o termo adicional tem a forma

$$F(s, s') = \gamma\Phi(s') - \Phi(s),$$

as políticas ótimas são preservadas sob as hipóteses apropriadas. Nos episódios desta nota, que encerram ao atingir um terminal, definimos $\Phi(s) = 0$ em **todos os estados terminais**. Para um episódio de T passos, os termos adicionais telescopam:

$$G'_0 = G_0 - \Phi(s_0) + \gamma^T \Phi(s_T) = G_0 - \Phi(s_0).$$

Para um mesmo estado inicial, o retorno muda por uma constante independente das ações, preservando a ordenação das políticas. Sem essa condição terminal, o termo residual pode mudar a preferência entre trajetórias. Em tarefas contínuas descontadas, potenciais limitados permitem que o termo residual desapareça no limite. Essa invariância do objetivo é distinta da convergência do algoritmo: o shaping não elimina as condições de visitas e taxas de aprendizado, nem a possibilidade de divergência com aproximação de funções. A aprendizagem pode ficar mais rápida ou mais lenta.

O *reward shaping* baseado em potencial preserva a política ótima, mas isso não significa que ele sempre acelere o aprendizado. Uma boa função potencial pode reduzir bastante o tempo de convergência. Uma função potencial ruim pode fazer o oposto: embora a política ótima final seja preservada, o agente pode demorar mais para aprender. Essa observação é importante pedagogicamente. O *reward shaping* baseado em potencial é útil porque permite incorporar conhecimento de domínio sem, em princípio, distorcer o objetivo final do agente. Mas ele não elimina a necessidade de modelagem cuidadosa: projetar uma boa função potencial continua sendo um problema heurístico, muito parecido com o projeto de boas heurísticas em busca informada.

8.4 Exemplo no Mundo Grade

Para tornar a ideia concreta, consideremos novamente o Mundo Grade. Suponha que queiramos definir uma função potencial simples que expresse a intuição de que estados mais próximos do objetivo positivo parecem mais promissores. Uma escolha natural é usar a distância de Manhattan até o estado terminal com recompensa $+1$. Assim, podemos definir uma função potencial $\Phi(s)$ que aumenta entre estados não terminais à medida que o agente se aproxima do objetivo. Por exemplo, podemos usar uma versão normalizada da forma:

$$\Phi(s) = \begin{cases} 0, & s \text{ terminal;} \\ 1 - \frac{d_{\text{Manhattan}}(s, \text{goal})}{D_{\text{max}}}, & s \text{ não terminal.} \end{cases}$$

onde D_{max} é a maior distância de Manhattan relevante na grade. Entre os estados não terminais, os mais próximos do objetivo têm maior potencial. Os dois terminais têm potencial zero. Na transição para um terminal, $F(s, s') = -\Phi(s)$; esse ajuste compensa os sinais intermediários e preserva o retorno até a constante inicial.

³Ng, Harada e Russell (1999), Policy invariance under reward transformations.

A partir dessa função, o termo de *reward shaping* baseado em potencial é:

$$F(s, s') = \gamma\Phi(s') - \Phi(s)$$

Para transições entre estados não terminais, o incentivo é positivo quando $\gamma\Phi(s') > \Phi(s)$; portanto, a aproximação ao objetivo deve ser suficiente para compensar o desconto. O sinal não deve ser interpretado apenas comparando os potenciais sem desconto.

Exemplo numérico

Considere, para simplificar, $\gamma = 0,9$. Suponha, para ilustrar a fórmula, dois estados não terminais com potenciais hipotéticos. Em s , o potencial é

$$\Phi(s) = 0,40$$

e que, após executar uma ação, o agente transite para um estado s' mais próximo do objetivo, com

$$\Phi(s') = 0,70$$

Então o termo adicional de *reward shaping* será:

$$F(s, s') = 0,9 \times 0,70 - 0,40 = 0,63 - 0,40 = 0,23$$

Ou seja, essa transição recebe um reforço adicional positivo de 0,23.

Agora suponha o caso oposto: o agente se move para um estado menos promissor, com

$$\Phi(s') = 0,20$$

Nesse caso,

$$F(s, s') = 0,9 \times 0,20 - 0,40 = 0,18 - 0,40 = -0,22$$

A transição recebe, portanto, um *shaping* negativo, que desencoraja esse tipo de movimento.

O agente continua aprendendo a maximizar recompensa futura, mas agora não precisa esperar apenas pela recompensa terminal para começar a distinguir movimentos promissores de movimentos pouco úteis. Transições que parecem aproximá-lo do objetivo passam a carregar um pequeno sinal positivo; transições que parecem afastá-lo passam a carregar um pequeno sinal negativo. Isso pode acelerar bastante o aprendizado inicial em ambientes com recompensa esparsa.

Esse exemplo também revela uma limitação importante. A função potencial acima considera apenas a proximidade do objetivo positivo. Ela não leva em conta, por exemplo, a presença do estado terminal negativo ou de obstáculos. Assim, embora seja uma heurística razoável, ela pode ser imperfeita. Em certos estados, aproximar-se do objetivo pode também significar aproximar-se de uma região perigosa. Isso mostra que projetar uma boa função potencial é um problema heurístico: quanto melhor a função capturar a estrutura real do ambiente, maior a chance de o *reward shaping* ajudar; quanto pior ela capturar essa estrutura, maior o risco de induzir um viés enganoso durante o aprendizado.

8.5 Limitações

O *reward shaping* pode tornar o aprendizado muito mais eficiente, sobretudo em ambientes com recompensas esparsas. No entanto, ele não deve ser visto como uma solução automática ou universal. Sua utilidade depende fortemente da qualidade do conhecimento de domínio embutido na função adicional de recompensa. Em outras palavras: *reward shaping* ajuda quando a heurística usada aponta, de fato, na direção de comportamentos úteis; quando essa heurística é ruim, o shaping pode introduzir viés e atrapalhar o processo de aprendizagem.

Mesmo no caso baseado em potencial, em que a política ótima final é preservada, ainda existe um risco importante: a função potencial pode ser *enganosa do ponto de vista do tempo de convergência*. Uma função potencial razoável pode acelerar bastante o aprendizado inicial, mas uma função ruim pode fazer o agente demorar mais para aprender do que demoraria sem shaping. Assim, a garantia teórica da forma potencial-based é limitada: ela protege a política ótima final, mas não garante que o caminho até ela será mais curto.

O exemplo do Mundo Grade ilustra bem esse ponto. Uma função potencial baseada apenas na distância até o objetivo positivo parece intuitiva, mas pode ignorar aspectos importantes do ambiente, como a presença de um estado terminal negativo ou de obstáculos. Nesses casos, a heurística pode premiar transições que aproximam o agente do objetivo, mesmo quando essa aproximação também aumenta o risco de uma transição muito ruim. O problema não é a forma matemática do *reward shaping*, mas a qualidade da informação heurística que foi colocada na função potencial.

Se a função potencial refletisse perfeitamente o valor real de cada estado, o problema de aprendizado ficaria quase resolvido desde o início. Mas construir uma função assim é, em geral, tão difícil quanto projetar uma heurística quase perfeita em busca informada. Por isso, o *reward shaping* deve ser entendido como uma ajuda heurística, não como um atalho que elimina a dificuldade do problema. Há também um limite conceitual importante. Quanto mais sofisticada e específica for a função potencial, mais conhecimento humano de domínio estará sendo embutido no sistema. Isso pode ser desejável em muitos contextos práticos, mas também reduz a generalidade da solução: uma função potencial cuidadosamente ajustada para um ambiente particular pode não se transferir bem para variações do mesmo problema. Assim como na engenharia de características, há um compromisso entre *ajudar o agente* e *manter a solução geral*.

Em resumo, o *reward shaping* é uma ferramenta útil para acelerar o aprendizado, especialmente quando as recompensas do ambiente são escassas ou tardias. Mas seu sucesso depende da mesma qualidade que sustenta tantas outras heurísticas em IA: a capacidade de capturar, de forma simples e imperfeita, aspectos realmente relevantes da estrutura do problema.

O *reward shaping* é especialmente útil quando o problema original fornece recompensas esparsas ou muito tardias, tornando o aprendizado inicial lento. Nesses casos, um sinal intermediário bem projetado pode orientar a exploração e acelerar a propagação de valor. A forma baseada em potencial é particularmente importante porque permite introduzir esse conhecimento heurístico sem alterar, em princípio, a política ótima final. Ainda assim, o sucesso do método depende da qualidade da função potencial: uma boa heurística pode acelerar bastante a convergência, enquanto uma heurística ruim pode atrasá-la.

Pergunta 10

Considere o Mundo Grade com um estado terminal positivo, um estado terminal negativo e transições estocásticas. Proponha uma função potencial simples $\Phi(s)$ para usar em *reward shaping* baseado em potencial. Em seguida, discuta: (a) que tipo de comportamento essa função tenderia a incentivar; (b) que aspecto importante do ambiente ela possivelmente ignoraria; e (c) por que, mesmo sendo imperfeita, ela ainda pode ser útil para acelerar o aprendizado.

9 Aprendizado por Reforço com Feedback Humano

Os conceitos de política, recompensa e aproximação de funções também aparecem no treinamento de modelos de linguagem de grande porte (*Large Language Models*, ou LLMs). Um caso importante é o **Aprendizado por Reforço com Feedback Humano** (*Reinforcement Learning from Human Feedback*, ou RLHF).

Durante seu treinamento inicial, um modelo de linguagem aprende, essencialmente, a prever o próximo token a partir dos tokens anteriores. Esse objetivo permite que o modelo aprenda regularidades linguísticas e conhecimentos presentes nos dados, mas não garante que suas respostas sejam úteis, corretas, seguras ou compatíveis com as instruções do usuário. Produzir uma continuação provável não é necessariamente o mesmo que produzir uma resposta desejável.

O RLHF tenta reduzir essa diferença usando avaliações humanas como fonte de informação sobre o comportamento desejado. Em vez de exigir que uma pessoa escreva uma função matemática capaz de atribuir recompensa a qualquer resposta possível, o método coleta comparações entre respostas produzidas pelo modelo. Essas comparações são usadas para aprender uma função de recompensa aproximada.

9.1 De preferências humanas a uma função de recompensa

Considere um prompt x e duas respostas candidatas y_A e y_B . Um avaliador humano pode indicar que prefere y_A a y_B :

$$y_A \succ y_B.$$

Essa comparação não fornece diretamente uma recompensa numérica. Ela fornece apenas uma relação de preferência. A partir de muitas comparações desse tipo, treina-se um *modelo de recompensa*:

$$\hat{r}_\phi(x, y),$$

onde x é o prompt, y é uma resposta e ϕ representa os parâmetros do modelo. O valor $\hat{r}_\phi(x, y)$ é uma estimativa escalar de quanto a resposta y parece satisfazer as preferências observadas durante o treinamento.

O modelo de recompensa é, portanto, uma função aproximada. Assim como a aproximação linear de $Q(s, a)$ estudada anteriormente, ele precisa generalizar a partir de exemplos limitados. Avaliadores humanos não conseguem examinar todas as respostas que um LLM pode produzir. O modelo de recompensa deve aprender regularidades nas comparações disponíveis e aplicá-las a respostas nunca avaliadas diretamente.

Essa generalização é necessária, mas introduz um risco. O modelo de recompensa não representa perfeitamente as preferências humanas. Ele representa apenas o que conseguiu aprender a partir dos exemplos fornecidos.

Pergunta 11

Por que não pedir que avaliadores humanos atribuam diretamente uma recompensa a cada resposta produzida durante o treinamento e durante o uso do LLM? Que papel o modelo de recompensa desempenha na escalabilidade do processo?

9.2 O processo de RLHF

Uma forma clássica de RLHF utiliza três etapas principais:

1. **Ajuste supervisionado.** O modelo inicial é treinado com exemplos de prompts acompanhados de respostas desejáveis. Essa etapa produz uma política inicial, frequentemente denominada π_{ref} ou política de referência.
2. **Aprendizado do modelo de recompensa.** Para um mesmo prompt, o modelo produz diferentes respostas. Avaliadores humanos ordenam ou comparam essas respostas. As preferências coletadas são usadas para ajustar $\hat{r}_\phi(x, y)$.
3. **Otimização da política.** O modelo de linguagem é tratado como uma política π_θ . Seus parâmetros são ajustados para aumentar a recompensa prevista por $\hat{r}_\phi$.

Somente a terceira etapa corresponde propriamente ao aprendizado por reforço. A primeira usa aprendizado supervisionado. A segunda também constitui um problema supervisionado, pois o modelo de recompensa aprende a partir das comparações fornecidas pelos avaliadores.

Se a política fosse ajustada apenas para maximizar a recompensa prevista, ela poderia afastar-se excessivamente do modelo de referência e explorar falhas do modelo de recompensa. Por isso, a otimização costuma considerar dois objetivos:

aumentar a recompensa prevista e evitar mudanças excessivas na política.

Uma formulação conceitual desse compromisso é:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [\hat{r}_\phi(x, y)] - \beta D_{\text{KL}}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)).$$

O primeiro termo favorece respostas com recompensa prevista elevada. O segundo penaliza o afastamento em relação à política de referência. A divergência D_{KL} mede a diferença entre as distribuições de probabilidade produzidas pelas duas políticas, enquanto β controla a intensidade da penalidade.

Não estudaremos o algoritmo usado para resolver essa otimização. O ponto central é que a política do LLM é ajustada para aumentar uma recompensa aprendida, mantendo alguma proximidade com o comportamento adquirido durante o ajuste supervisionado.

9.3 RLHF como um problema de aprendizado por reforço

A geração de texto pode ser representada como um processo sequencial. A cada passo, o modelo observa o prompt e os tokens já produzidos, escolhe o próximo token e acrescenta esse token à resposta.

A Tabela 2 relaciona os elementos desse processo aos conceitos de aprendizado por reforço.

Conceito de AR	Interpretação na geração de texto
Estado s_t	Prompt e sequência de tokens já produzida até o instante t
Ação a_t	Próximo token escolhido pelo modelo
Política $\pi_\theta(a_t s_t)$	Distribuição de probabilidades sobre os possíveis próximos tokens
Transição	A ação escolhida é acrescentada à sequência, produzindo o próximo estado
Episódio	Geração completa de uma resposta
Recompensa	Avaliação humana ou pontuação fornecida pelo modelo de recompensa

Tabela 2: Correspondência entre os componentes de um problema de aprendizado por reforço e a geração de texto por um LLM.

Nessa formulação, a função de transição é simples: o token escolhido é acrescentado ao texto já existente. A principal incerteza está na recompensa. Não existe uma função conhecida que determine exatamente quão útil, correta ou segura é qualquer resposta possível. O RLHF tenta aprender uma aproximação dessa função a partir de preferências humanas.

Também é possível adotar uma abstração mais simples e tratar a resposta completa como uma única ação. Nesse caso, o prompt é o contexto, cada resposta possível é uma ação e a recompensa é atribuída ao final. Essa formulação se aproxima de um *bandit* contextual. A formulação token a token é mais fiel ao processo de geração, mas torna explícito o problema de atribuição de crédito: se uma resposta recebe uma avaliação ruim, quais tokens ou decisões tomadas durante sua geração foram responsáveis pelo resultado?

É importante observar que o RLHF não utiliza necessariamente Q-learning ou SARSA. Esses algoritmos aprendem valores de estado-ação a partir de alvos de Diferença Temporal. No RLHF clássico, costuma-se ajustar diretamente a política do modelo de linguagem. Apesar dessa diferença algorítmica, permanecem os mesmos elementos fundamentais: política, interação, recompensa, aproximação de funções e otimização do comportamento esperado.

9.4 Exemplo resolvido: aprendizado de preferências sobre respostas

Considere o seguinte prompt:

Explique o que é overfitting em no máximo duas frases.

Um modelo de linguagem produz três respostas:

- y₁:** *Overfitting ocorre quando um modelo se ajusta excessivamente aos dados de treinamento, incluindo seus ruídos e particularidades. Com isso, ele apresenta bom desempenho no treinamento, mas generaliza mal para novos dados.*
- y₂:** *Overfitting ocorre quando um modelo aprende muito bem os dados de treinamento. Ele pode inclusive memorizar particularidades desses dados. Seu desempenho no treinamento costuma ser elevado. Entretanto, o desempenho em novos dados pode ser ruim. Técnicas de regularização podem ajudar a reduzir o problema.*
- y₃:** *Overfitting ocorre quando o modelo é simples demais para aprender os padrões existentes nos dados. Por isso, ele apresenta desempenho ruim tanto no treinamento quanto no teste.*

A primeira resposta é correta, clara e respeita o limite de duas frases. A segunda também é conceitualmente correta, mas não segue a restrição de tamanho. A terceira respeita o tamanho, porém descreve *underfitting*, não *overfitting*. Consider que, com base nessa análise, um avaliador humano estabelece a seguinte preferência:

$$y_1 \succ y_2 \succ y_3.$$

Para tornar o exemplo transparente, suponha um modelo de recompensa linear com três características:

$$\hat{r}(x, y) = 2f_{\text{cor}}(x, y) + f_{\text{inst}}(x, y) + 0,5f_{\text{clar}}(x, y),$$

onde:

- f_{cor} representa a correção conceitual estimada;
- f_{inst} representa o cumprimento da instrução;
- f_{clar} representa a clareza da resposta.

Na prática, modelos de recompensa não precisam utilizar características explicitamente definidas por seres humanos. Eles podem aprender representações internas a partir dos próprios textos. O modelo linear é usado aqui apenas para tornar o cálculo visível.

Suponha que o modelo atribua os seguintes valores:

Resposta	f_{cor}	f_{inst}	f_{clar}	$\hat{r}(x, y)$
y_1	1	1	0,8	3,40
y_2	1	0	0,9	2,45
y_3	0	1	0,7	1,35

Para y_1 , temos:

$$\hat{r}(x, y_1) = 2(1) + 1 + 0,5(0,8) = 3,40.$$

Para y_2 :

$$\hat{r}(x, y_2) = 2(1) + 0 + 0,5(0,9) = 2,45.$$

Para y_3 :

$$\hat{r}(x, y_3) = 2(0) + 1 + 0,5(0,7) = 1,35.$$

Portanto:

$$\hat{r}(x, y_1) > \hat{r}(x, y_2) > \hat{r}(x, y_3).$$

O modelo de recompensa reproduziu a ordem estabelecida pelos avaliadores. Durante a etapa de aprendizado por reforço, respostas semelhantes a y_1 receberão recompensa prevista maior. A otimização tenderá, portanto, a aumentar sua probabilidade. Respostas semelhantes a y_2 ou y_3 tenderão a perder probabilidade.

Observe que o agente não recebeu uma resposta correta para copiar durante essa etapa. Ele recebeu um sinal escalar que indica quais respostas parecem preferíveis. Esse sinal orienta a modificação da política.

Quando a recompensa aproximada falha

Suponha agora que o modelo produza uma quarta resposta:

y_4 : *Overfitting ocorre sempre que a acurácia de treinamento é maior que a acurácia de teste. O problema pode ser completamente eliminado aumentando o conjunto de treinamento.*

A resposta é curta, clara e usa termos associados ao tema. Entretanto, suas duas afirmações são excessivas: uma diferença entre treinamento e teste não caracteriza, por si só, qualquer caso de *overfitting*, e aumentar o conjunto de treinamento não garante a eliminação completa do problema.

Suponha que, apesar disso, o modelo de recompensa interprete a linguagem confiante e o vocabulário técnico como evidências de correção:

$$f_{\text{cor}}(x, y_4) = 1, \quad f_{\text{inst}}(x, y_4) = 1, \quad f_{\text{clar}}(x, y_4) = 1.$$

Nesse caso:

$$\hat{r}(x, y_4) = 2(1) + 1 + 0,5(1) = 3,50.$$

Logo:

$$\hat{r}(x, y_4) > \hat{r}(x, y_1),$$

embora avaliadores humanos bem informados prefiram y_1 .

Se a política explorar sistematicamente essa falha, poderá aprender a produzir respostas confiantes e tecnicamente plausíveis, mas incorretas. O algoritmo de otimização não

cometeu um erro: ele aumentou a recompensa que recebeu. O problema está no fato de que a recompensa aprendida é apenas uma aproximação do objetivo humano.

Esse fenômeno é denominado *exploração da recompensa* ou *reward hacking*. O agente encontra uma forma de aumentar a recompensa prevista sem realizar adequadamente aquilo que o projetista pretendia incentivar.

Pergunta 12

No exemplo anterior, o modelo de linguagem aprendeu a produzir respostas como y_4 , que recebem recompensa elevada apesar de conterem afirmações incorretas. Analise o problema respondendo: **(a)** por que a política está agindo racionalmente em relação à recompensa recebida; **(b)** que erro de generalização foi cometido pelo modelo de recompensa; **(c)** que novos exemplos de preferência poderiam ajudar a corrigir esse erro; e **(d)** por que aumentar a capacidade do modelo de linguagem, sem modificar o modelo de recompensa, poderia agravar o problema.

9.5 Limitações e cuidados

O RLHF permite transmitir objetivos difíceis de expressar por meio de uma função de recompensa escrita manualmente. Em muitos casos, é mais fácil para uma pessoa comparar duas respostas do que atribuir uma pontuação absoluta e precisa a cada uma delas. Ainda assim, o método possui limitações importantes.

Primeiro, as preferências coletadas dependem dos avaliadores, das instruções que eles recebem e dos exemplos apresentados. Pessoas diferentes podem discordar sobre clareza, utilidade, segurança ou adequação. O modelo de recompensa aprende uma aproximação das preferências presentes nos dados, não uma definição universal de comportamento correto.

Segundo, o modelo de recompensa precisa avaliar respostas diferentes das que apareceram em seu treinamento. Como qualquer aproximador, ele pode generalizar de forma indevida. Quanto mais capaz for a política de buscar respostas com recompensa elevada, maior pode ser sua capacidade de encontrar regiões em que a aproximação falha.

Terceiro, a recompensa costuma ser atribuída à resposta completa. Isso torna difícil identificar quais decisões tomadas durante a geração foram responsáveis pela avaliação final. Trata-se novamente do problema de atribuição de crédito.

Por fim, uma recompensa prevista elevada não garante que a resposta seja verdadeira, segura ou útil. Ela garante apenas que o modelo de recompensa reconheceu naquela resposta características associadas a avaliações positivas. O RLHF não elimina o problema de especificar objetivos. Ele transfere parte desse problema para a coleta de preferências e para o aprendizado da função de recompensa.

9.6 RLHF e *reward shaping*

Embora ambos utilizem recompensas para orientar o comportamento, RLHF e *reward shaping* não são a mesma técnica.

No *reward shaping*, parte-se de uma recompensa já definida e adiciona-se um termo intermediário para acelerar o aprendizado. No RLHF, o problema central é diferente: a recompensa correspondente ao comportamento desejado não está disponível em forma matemática. Ela precisa ser aproximada a partir de comparações humanas.

Nos dois casos, porém, permanece a mesma advertência: o agente aprende a maximizar o sinal que recebe, não necessariamente a intenção que motivou a construção desse sinal. Uma recompensa mal projetada ou mal aproximada pode produzir comportamentos indesejados mesmo quando o algoritmo de aprendizado funciona corretamente.

Também existem métodos de alinhamento por preferências que não executam explicitamente a etapa de aprendizado por reforço. Esses métodos ajustam a política diretamente a partir de pares de respostas preferidas e rejeitadas. Portanto, nem todo treinamento com preferências humanas deve ser chamado, em sentido estrito, de RLHF. O RLHF é uma abordagem específica dentro de uma família mais ampla de métodos de alinhamento de modelos de linguagem.

Pergunta 13

Compare RLHF e *reward shaping*. Em ambos os casos, de onde vem a informação humana incorporada à recompensa? Qual técnica procura acelerar o aprendizado sem alterar a política ótima e qual procura aprender uma aproximação de um objetivo que não foi fornecido explicitamente?

10 Resumo

A tabela abaixo sintetiza as principais características dos algoritmos de AR estudados nesta aula.

Aspecto	Q-learning tabular	SARSA tabular	Q-learning aprox. linear
O que é aprendido	Uma entrada $Q(s, a)$ por par estado-ação	Uma entrada $Q(s, a)$ por par estado-ação	n pesos $\mathbf{w}$ compartilhados entre todos os pares
Tipo de política	<i>Off-policy</i>	<i>On-policy</i>	<i>Off-policy</i>
Alvo de atualização	$r + \gamma \max_{a'} Q(s', a')$	$r + \gamma Q(s', a')$	$r + \gamma \max_{a'} Q(s', a')$
Memória necessária	$ S \times A $ entradas	$ S \times A $ entradas	n pesos ($n \ll S \times A $)
Generalização	Nenhuma: cada par é independente	Nenhuma: cada par é independente	Automática via características compartilhadas
Convergência	Para Q^* sob as condições tabulares	Para Q^π com política fixa; para Q^* com GLIE e condições tabulares	Pode divergir, mesmo com aproximação linear
Escalabilidade	Impraticável para $ S $ grande	Impraticável para $ S $ grande	Praticável se n for pequeno
Perto de regiões perigosas	Ignora risco de exploração	Leva em conta risco de exploração	Depende das características
Regra de atualização	$Q(s, a) \leftarrow Q(s, a) + \alpha \delta$	$Q(s, a) \leftarrow Q(s, a) + \alpha \delta$	$w_i \leftarrow w_i + \alpha \delta f_i(s, a)$

O AR aborda o problema de aprender a agir a partir de experiências para maximizar recompensas acumuladas. No cenário livre de modelo estudado, T e R são desconhecidos; o agente aprende valores ou políticas sem estimar explicitamente o modelo. Esse cenário é formalizado como um PDM parcialmente especificado. Q-learning e SARSA são dois algoritmos TD que aprendem Q -valores diretamente das transições, usando a equação de Bellman como alvo de atualização amostrada. A diferença entre eles é precisa e consequente: o Q-learning é *off-policy* (usa $\max_{a'}$ no alvo) e o SARSA é *on-policy* (usa a ação realmente escolhida). O dilema exploração-aproveitamento é estrutural e deve ser tratado explicitamente. A aproximação linear permite operar em espaços de estados grandes por meio da generalização, mas não herda a garantia de convergência do método tabular.

Pergunta 14

O Alfa-Beta (nas notas de Busca Adversarial) garante a mesma resposta que o Minimax completo. O Q-learning tabular converge para Q^* sob as condições estabelecidas. O Q-learning com aproximação linear *não* oferece essa garantia. Por que aceitamos uma abordagem sem garantia teórica? O que ganhamos e o que perdemos? Em quais domínios a aproximação linear seria claramente insuficiente?

A Figura 13 resume o funcionamento do aprendizado por reforço baseado em valor como um ciclo fechado.

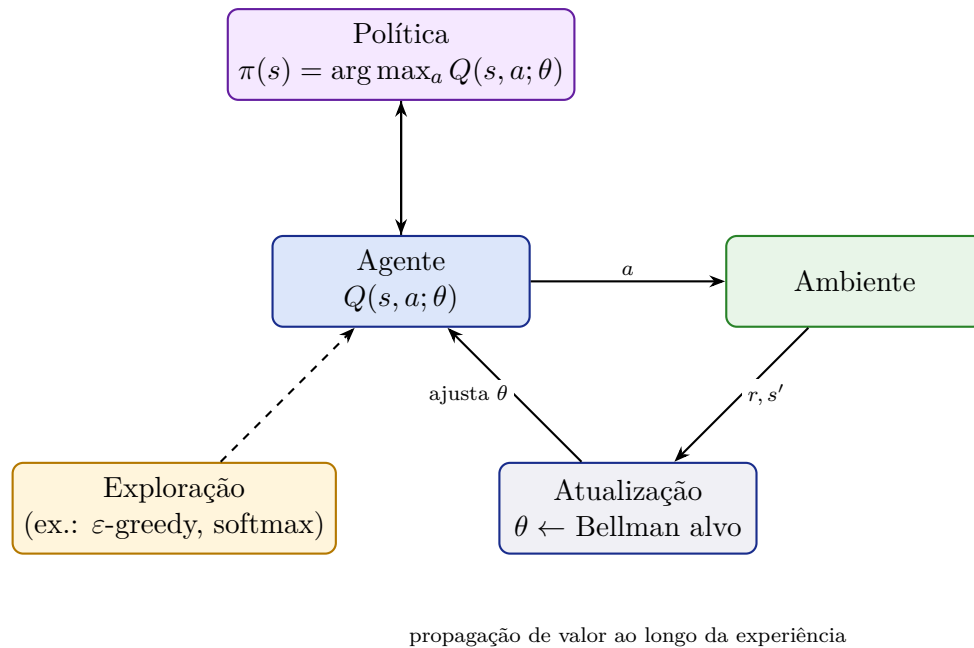


Figura 13: Visão integrada do aprendizado por reforço baseado em valor. O agente mantém uma função de valor de ação $Q(s, a; \theta)$ (tabular ou aproximada), interage com o ambiente e atualiza seus parâmetros com base em alvos derivados da equação de Bellman. A política é obtida a partir de Q , enquanto a exploração garante diversidade de experiência.

11 Atividades Intra-Classe

As atividades abaixo devem ser resolvidas em grupo, com foco em explicar o raciocínio passo a passo, e não apenas em obter a resposta final.

11.1 Aquecimento

Antes das atividades principais, responda rapidamente:

- Um agente observa a transição $(s, \text{Norte}, r = -0,04, s')$. Qual é o alvo de atualização do Q-learning para $Q(s, \text{Norte})$, supondo $\gamma = 0,9$ e $\max_{a'} Q(s', a') = 0,8$?
- Qual seria o alvo de atualização do SARSA para a mesma transição, supondo que a política ε -greedy escolheu $a' = \text{Sul}$ em s' com $Q(s', \text{Sul}) = 0,3$?
- Por que $Q(s, a)$ inicializado com zero é uma escolha razoável para o início do treinamento?
- Qual a diferença entre Monte Carlo e TD no que diz respeito ao momento em que os valores são atualizados?
- Um agente usa $\alpha = 1,0$ durante todo o treinamento em um ambiente estocástico. Que problema isso pode causar?
- Qual a diferença entre “exploração” e “aproveitamento” no contexto do Q-learning? Dê um exemplo de cada na situação de um agente no Mundo Grade.

Síntese oral. Explique o Q-learning e o SARSA para um colega usando apenas:

- a ideia de tentativa e erro;
- a noção de “ajustar expectativas conforme a experiência”;
- o conceito de recompensa futura descontada;
- a diferença entre “aprender como você *deveria* agir” e “aprender como você *de fato* age”.

Evite equações. Se você não consegue explicar sem fórmulas, provavelmente ainda não internalizou o conceito.

11.2 Q-learning tabular no Mundo Grade

Considere o Mundo Grade 4×3 com $\gamma = 0,9$, $\alpha = 0,5$ e *living reward* de $-0,04$ em todos os estados não terminais. Assuma que todos os $Q(s, a)$ são inicializados com 0.

- Simule manualmente o seguinte episódio, mostrando os cálculos completos para cada atualização. Use apenas as transições indicadas (ignore a estocasticidade):
 $(1, 1) \xrightarrow{\text{Norte}} (1, 2), r = -0,04$; $(1, 2) \xrightarrow{\text{Norte}} (1, 3), r = -0,04$; $(1, 3) \xrightarrow{\text{Leste}} (2, 3), r = -0,04$; $(2, 3) \xrightarrow{\text{Leste}} (3, 3), r = -0,04$; $(3, 3) \xrightarrow{\text{Leste}} (4, 3) \text{ (Terminal}^{+1}), r = +1$.
- Quais entradas da tabela Q foram alteradas após esse episódio? Qual é o valor de $Q((1, 1), \text{Norte})$?
- Execute o mesmo episódio uma segunda vez (com os valores Q obtidos no item anterior). Como os valores se alteram? O que você observa sobre a velocidade de propagação dos valores até $(1, 1)$?
- Por que os valores próximos ao estado terminal convergem mais rápido do que os valores distantes dele?

11.3 Comparação entre Q-learning e SARSA

Utilize o Mundo Grade com $\gamma = 0,9$, $\alpha = 0,5$, $\varepsilon = 0,1$. Considere o agente em $s = (3, 2)$ executando Norte, chegando a $s' = (3, 3)$ com $r = -0,04$. Em s' , a política ε -greedy sorteia a ação exploratória $a' = \text{Leste}$, com $Q((3, 3), \text{Leste}) = 0,20$. A ação gulosa nas estimativas atuais em s' tem $\max_{a'} Q((3, 3), a') = 0,60$. Antes da atualização, $Q((3, 2), \text{Norte}) = 0,10$; use esse mesmo valor inicial nos dois algoritmos.

- Calcule a atualização de $Q((3, 2), \text{Norte})$ pelo Q-learning. Mostre o alvo e o erro δ .
- Calcule a atualização de $Q((3, 2), \text{Norte})$ pelo SARSA com $a' = \text{Leste}$. Mostre o alvo e o erro δ .
- Compare os dois resultados. Qual algoritmo produziu a estimativa maior? O que isso revela sobre a diferença entre *on-policy* e *off-policy*?
- Suponha que o agente esteja treinando em um ambiente real (robô físico) onde cair em $(4, 2)$ tem consequências físicas reais. Qual algoritmo você recomendaria? Justifique.

11.4 Q-learning com aproximação linear

Considere o Pac-Man com duas características: $f_1(s, a) = 1/(1 + d_{\text{pastilha}}(s, a))$ e $f_2(s, a) = 1/(1 + d_{\text{fantasma}}(s, a))$. As distâncias são medidas a partir da posição que resultaria do movimento a , com os demais elementos mantidos como observados em s ; assim, as features distinguem ações e são finitas mesmo para $d = 0$. Se não houver pastilhas (ou fantasmas), a feature correspondente vale zero. Defina $Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a)$. Pesos iniciais: $w_1 = 0$, $w_2 = 0$. Use $\alpha = 0,1$, $\gamma = 0,9$.

- Transição 1:** para a ação Leste em s , as distâncias projetadas são 2 células até a pastilha ($f_1 = 1/3$) e 5 até o fantasma ($f_2 = 1/6$). O agente executa Leste, recebe $r = +0,5$, e s' não é terminal. Com os pesos ainda zerados, $\max_{a'} Q(s', a') = 0$. Calcule $Q(s, \text{Leste})$ antes da atualização, δ e os pesos atualizados.
- Transição 2:** para a ação Norte em s'' , as distâncias projetadas são 1 célula até a pastilha e 1 até o fantasma ($f_1 = f_2 = 1/2$). Use os pesos do item anterior. O agente executa Norte, recebe $r = -500$ (morre) e s''' é terminal. Calcule $Q(s'', \text{Norte})$ antes da atualização, δ e os pesos resultantes.
- Após as duas transições, interprete as correções nos pesos. Por que a redução de ambos não permite concluir que ambas as características causaram a morte?

11.5 Exploração e arrependimento

- Um agente usa ε -greedy com $\varepsilon = 0,1$ no Mundo Grade: são 11 estados, dos quais dois terminais, e quatro ações em cada um dos nove estados não terminais (36 pares). Em cada passo exploratório, a ação é sorteada uniformemente entre as quatro. Se $\mathbb{E}[N_s]$ é o número esperado de decisões no estado s , expresse o número esperado de seleções exploratórias do par (s, a) . Depois, calcule-o para 10.000 decisões, sob a hipótese adicional de ocupação uniforme dos nove estados não terminais. Essa ocupação uniforme é uma simplificação do exercício, não uma consequência de ε -greedy.
- Um segundo agente usa uma função de exploração $f(u, n) = u + 100/\max(1, n)$, usada apenas na seleção de ações. Descreva qualitativamente como a distribuição de visitas entre os pares (s, a) diferirá do agente ε -greedy ao longo do treinamento, especialmente nas primeiras iterações.
- É possível ordenar o arrependimento total desses agentes apenas com essas informações? Discuta condições sobre o ambiente, as estimativas iniciais e o bônus em que cada estratégia poderia funcionar melhor.

11.6 Perguntas para discussão em sala

Pergunta 15

Sob quais condições tabulares e de exploração SARSA e Q-learning convergem para Q^* ? Explique por que $\varepsilon \rightarrow 0$ sozinho não basta, e como empates entre ações ótimas podem produzir políticas diferentes com o mesmo valor. Que implicação isso tem para a relação teórica entre os dois algoritmos?

Pergunta 16

O Q-learning tabular converge para Q^* sob as condições estabelecidas, mas o Q-learning aproximado não oferece essa garantia. Além disso, a aproximação linear assume que $Q(s, a)$ pode ser bem representado por uma combinação linear de características. Descreva um cenário concreto no Pac-Man em que essa suposição falha: dois estados (s_1, a) e (s_2, a) com características quase idênticas mas valores Q muito diferentes. O que acontece com o aprendizado nesses casos?

Pergunta 17

Considere um agente que treina no Mundo Grade com ε -greedy ($\varepsilon = 0,1$) por 100.000 episódios e depois é testado sem exploração ($\varepsilon = 0$). Por que o desempenho no teste pode diferir do desempenho durante o treinamento? Que papel o *living reward* e o fator de desconto desempenham nessa diferença?

Pergunta 18

O AR tem sido aplicado com sucesso em jogos (Atari, Go, xadrez via AlphaZero) e em robótica. Identifique um aspecto do AR que é essencial para jogos, mas problemático em robótica real, por exemplo, a segurança durante a exploração. Como o dilema exploração-aproveitamento se manifesta de forma diferente nesses dois domínios? O SARSA seria mais adequado do que o Q-learning para robótica? Por quê?

Referências

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992.